

THE DURABLE7 FIELD GUIDE

Persistent Data Structures

Nine languages, one contract,
forty-odd immutable collections

A complete tour of the collections in the
Durable7 repository — what each one is,
why it exists, and when to reach for it.

Pre-release 0.1.0 • MIT No Attribution

Durable7: The Persistent Data Structure Field Guide

This document describes the collections implemented in the Durable7 repository. It is a companion to — not a replacement for — the workspace API specifications and public headers, which remain the normative source for contracts, complexity, allocation behavior, and validation evidence. Where this guide and a workspace specification disagree, the specification wins and this guide has a bug.

The repository ports every family to **C**, **C++**, **C#**, **Haskell**, **Kotlin**, **OCaml**, **Python**, **Rust**, and **TypeScript**. Code samples are drawn from whichever port makes the point most clearly; every behavior described in the shared-contract sections holds in all nine unless the text says otherwise.

Set in Libertinus with Inconsolata for code; diagrams drawn in TikZ; typeset with L^AT_EX.

Licensed **MIT No Attribution (MIT-0)**. Use it, fork it, ship it.

Contents

How to read this guide	vii
I Foundations	1
1 What Persistence Buys	3
1.1 Path copying, and the shape of an update	3
1.2 The version DAG	4
1.3 What it costs	5
1.4 Where mutation is allowed	6
1.5 Why nine implementations	6
2 The Shared Contract	9
2.1 Vocabulary	9
2.2 Policies are values, and they are retained	10
2.3 Representatives	11
2.4 No-op identity	11
2.5 Failure atomicity	11
2.6 Ordering, stated honestly	12
2.7 Ownership, nine ways	12
II The Unordered World	15
3 CHAMP: The Hash Trie at the Center	17
3.1 From HAMT to CHAMP	18
3.2 Equality and diff without a lookup pass	19
3.3 Collisions	20
3.4 Single-descent factories	20
3.5 Sets, and what changes	21

3.6	Nine ports, one representation	22
4	Companions of the Trie	25
4.1	Persistent hash bag	25
4.2	Persistent bidirectional map	26
4.3	Set-valued multimap and relation	27
4.4	Map patch	28
4.5	Directed graph	29
4.6	Indexed map	29
4.7	Persistent chunked bit set	30
5	Patricia: When the Key Is Its Own Hash	31
5.1	Path compression and the branching bit	32
5.2	The reason it exists: structural merge	33
6	Ctrie: Snapshots Without Stopping the World	35
6.1	How a constant-time snapshot is possible	36
6.2	Crossing back into the persistent world	36
6.3	Why only two ports have the real thing	37
7	Builders, Sessions, and the Frozen Tier	39
7.1	Construction-only bulk builders	40
7.2	The one-way editing session	40
7.3	The frozen tier, and why it has not shipped	42
III	Sequences	45
8	Finger Trees	47
8.1	The deque	48
8.2	The measured tree	49
9	Measures: Monoids as Connective Tissue	51
9.1	The idea	51
9.2	The measures that ship	52
9.3	How the nine ports spell it	53
10	RRB Vectors	55

10.1	Regular and relaxed branches	56
10.2	The representation boundary, stated up front	56
10.3	Choosing between three sequences	57
11	Ropes and Text	59
11.1	Three layers, one machine	60
11.2	Builders	60
12	The Range-Update Sequence	61
12.1	The invariant that makes it work	62
12.2	The algebra, and its laws	63
12.3	In use	63
13	The Reversible Deque	65
14	DABA Lite: The Deliberate Outlier	67
14.1	Six cursors and a bounded fixup	68
14.2	The parts that differ by runtime	68
14.3	What it will not do	69
14.4	Space, and a validator that knows nothing	69
IV	Order and Priority	71
15	Sorted Collections	73
16	Canonical Sorted Sets: Zip and Zip-Zip Trees	75
16.1	Shape from contents	76
16.2	Getting ranks right	76
17	Insertion Order as a First-Class Property	79
17.1	Two indexes, one meaning	80
18	Priority: Three Structures, Three Reasons	83
18.1	The measured priority queue	83
18.2	The Brodal–Okasaki heap	84
18.3	The priority search queue	85
19	Intervals	87

V	Authenticated Collections	89
20	The Merkle Search Tree	91
20.1	Shape from hashes, not from history	92
20.2	Two prerequisites, taken seriously	93
21	Blocks, Proofs, Synchronization, and Merge	95
21.1	Blocks and stores	95
21.2	Packs, and the transport boundary	96
21.3	Seven budgets	96
21.4	Proofs	97
21.5	Synchronization	97
21.6	Three-way merge	98
VI	Navigation	99
22	Cursors	101
22.1	Gap semantics	101
22.2	Which families have one	102
22.3	One focused representation, many checkpoints	103
VII	Reference	107
A	Complexity at a Glance	109
B	The Same Structure in Nine Languages	113
C	Roads Not Taken	117
C.1	Rejected outright	117
C.2	Postponed, with a named gate	118
C.3	Shipped as documentation rather than as a type	119
D	Further Reading	121
	Index	123

How to read this guide

There are two honest ways to use a book about data structures. You can read it straight through, which is what you want when you are trying to understand a design space. Or you can open it at the one page that answers today’s question, which is what you want at 4pm on a Thursday. This guide is arranged so that both work.

If you are reading straight through. Part I explains what persistence buys and what it costs, and sets out the vocabulary that the rest of the book leans on — policies, representatives, no-op identity, failure atomicity. Nothing after it will quite land without it. Parts II through V then walk the families in the order a curious reader would want them: the hash trie first (it is the substrate for a third of everything else), then sequences, then ordered and priority collections, then the authenticated tier. Part VI covers cursors, which cut across every family. The appendices are reference material.

If you are looking something up. Every structure gets a *specimen card* — a compact panel with its purpose, its complexity, its policy requirements, and its nine-language spellings — followed by prose that explains the representation and the decisions inside it. Appendix A is a single sorted complexity table across the whole library. Appendix B is a name-to-name index across the nine ports. The subject index at the back covers both concepts and identifiers.

Three kinds of panel appear throughout.

The contract

Normative behavior that every port upholds. If a port cannot honor one of these, it says so explicitly in its own API notes rather than quietly differing.

Caveat

The place where a claim stops being true. These are the paragraphs worth reading twice — most production surprises with persistent structures live exactly at the

edge of an amortization argument or a policy assumption.

Aside

History, provenance, and the occasional road not taken. Skippable, but they are where the design reasoning usually hides.

A note on honesty. This library is unusually careful about the difference between a bound it can prove and a bound it would like to have. Several structures ship as *checkpoints*: the observable semantics match the reference port exactly, but the representation is simpler and the asymptotic claim is weaker. Wherever that is the case, this guide says so in the same breath as the feature. A guide that only reported the good news would be a worse guide.

PART I

Foundations

What Persistence Buys

The old version is still there. That one sentence is the whole library, and everything else is an argument about how to afford it.

Here is a small program, and a large idea hiding inside it.

```
var v1 = PersistentHashMap<int, string>.Empty.SetItem(1, "one");  
var v2 = v1.SetItem(2, "two");  
var v3 = v2.SetItem(1, "uno");
```

After three lines you are holding three maps. Not three references to one map that has been mutated three times — three genuinely independent collections, each of which will answer questions about its own contents forever. `v1` still has exactly one entry. `v2` still maps 1 to "one". You can hand `v1` to another thread while a third thread builds `v4` from `v3`, and no lock is involved anywhere, because nothing that either thread can see is ever written to again.

The obvious way to get that behavior is to copy the map on every update, and the obvious problem with the obvious way is that it costs $O(n)$ time and $O(n)$ space per edit. A library of persistent collections is, in essence, a long and inventive answer to the question: *how little can we copy and still tell the truth?*

1.1 Path copying, and the shape of an update

The answer, in one picture, is Figure 1.1. Update-shaped operations copy the nodes along the route from the root to the point of change and reuse every other node by reference. For a tree of branching factor b and n elements, that is $\log_b n$ new nodes rather than n — for a 32-way trie holding a million entries, four small nodes instead of a million entries.

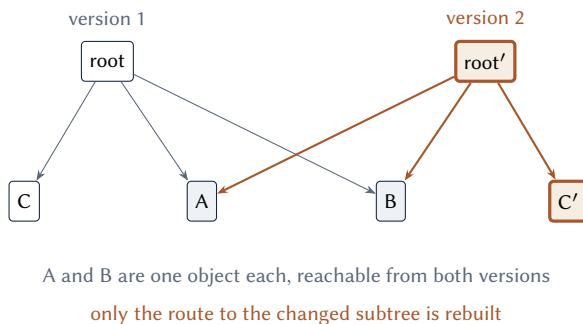


Figure 1.1: Path copying. Version 2 differs from version 1 in a single subtree. The two nodes on the route to that subtree are freshly allocated (copper); every other node is shared verbatim. Neither version can observe the other’s existence.

Two consequences follow immediately, and both matter more than they look.

Old versions cost almost nothing to keep. Retaining v1 does not pin a second copy of the data; it pins the handful of nodes that later versions replaced. A hundred snapshots of a slowly changing map are cheaper than one would guess and vastly cheaper than a hundred deep copies. This is what makes undo stacks, audit trails, and time-travel debugging practical rather than aspirational.

Readers need no synchronization. Every node reachable from a published root is immutable, so a reader that has captured a root is reading a frozen world. There is no torn read, no lock, no volatile fence in the read path, and no window during which a concurrent writer can make a traversal observe half of two different states. The synchronization problem does not get solved; it gets deleted.

1.2 The version DAG

Because an update produces a new value rather than modifying an old one, nothing forces versions into a line. Branch from v2 twice and you get two descendants that share v2’s structure and know nothing about each other (Figure 1.2). This is why persistent collections are the natural substrate for editors, transactional stores, speculative execution, and any system where “what if” has to be cheap.

That freedom has a price, and it is the single most important caveat in this book.

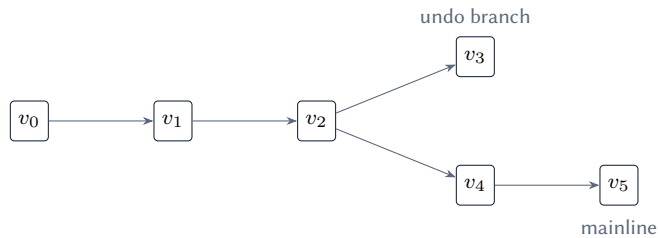


Figure 1.2: Versions form a directed acyclic graph, not a sequence. Every node in the picture is a live, complete collection.

Amortization does not survive branching

Amortized bounds are proved by a potential argument over *one linear history*: an expensive operation is paid for by cheap operations that preceded it. Branching breaks the argument. If a version sits just before an expensive rebalance, and you branch from it b times, each branch can trigger that same rebalance, and the accumulated credit pays for exactly one of them. Potential consumed by one child cannot pay for a sibling.

Durable7 therefore classifies every adaptive scheme before believing its amortized claim. Constant-size promotion thresholds are safe — re-branching re-pays only $O(1)$. Schemes that do $O(n)$ work at a threshold — relabeling, compaction, wholesale redistribution — are not, and the library either avoids them or states the worst case per produced version. Where a bound holds only along a linear lineage, this guide says “linear lineage” explicitly.

1.3 What it costs

Persistent collections are not free, and pretending otherwise makes for a bad guide. The honest ledger looks like this.

- **Allocation per update.** Every edit allocates its copied path. A tight loop of point updates on a persistent map will allocate far more than the same loop on a hash table. This is the motivation for the entire builder and editing-session tier (Chapter 7).
- **Indirection on reads.** A 32-way trie lookup is a handful of pointer hops with good branch prediction, but it is still pointer hops. A flat open-addressed table

wins on a hot small map, every time.

- ▶ **Cache behavior.** Node-based structures scatter. The library’s answer is chunking — ropes hold runs of elements in contiguous leaves, RRB vectors hold 32-element arrays, chunked bit sets hold 64-bit words — so that scans get array speed even though edits get tree behavior.
- ▶ **Small collections lose.** The regime in which a persistent map looks worst is the one most programs spend most of their time in: maps with three entries. The library records a size-tiered representation as a strong idea that is deliberately *not* shipped, because it has not yet been justified with benchmarks (Appendix C).

1.4 Where mutation is allowed

A library that is honest about persistence has to be equally honest about its exceptions. Durable7 confines mutation to four clearly labeled places, and every one of them is documented as mutable wherever it appears:

Construction-only bulk builders mutate unpublished trie nodes while building a collection from a sequence, then freeze. Nothing observable is ever mutated, because the nodes are not reachable until the freeze.

One-way editing sessions adopt a persistent root, accept a run of edits under single ownership, and publish once. Publication consumes the session.

The concurrent hash trie is a genuinely mutable lock-free map whose distinguishing feature is that `Snapshot()` is $O(1)$.

DabaLite is a mutable sliding-window aggregator. It is in the library because it reuses the monoid vocabulary, not because it is persistent; it explicitly is not.

1.5 Why nine implementations

Most collection libraries exist once, and a “port” is a family resemblance. Here, semantic parity is the product. The same ordering rules, the same first-representative retention, the same no-op identity, the same failure atomicity, and — for the authenticated tier — the same bytes on the wire, in C, C++, C#, Haskell, Kotlin, OCaml, Python, Rust, and TypeScript.

The interesting part is what *does not* have to match. Rust uses `Arc` and forbids `unsafe`; C# exposes `ReadOnlyList<T>`; OCaml returns `option` and `result`; C is type-erased with explicit ownership and fallible allocators; Haskell keeps its sessions in `IO`. Idioms

are local. Semantics are not.

The checkpoint idea

Nine simultaneous implementations of a subtle algorithm would normally mean nine opportunities to quietly claim a bound nobody proved. Durable7's device for avoiding that is the *checkpoint port*: a port that preserves the observable API semantics and passes the same behavioral tests, while documenting exactly which representation or asymptotic property it does *not* yet have. OCaml's range-update sequence is a checkpoint — same algebra, same persistence, no implicit-AVL logarithmic lazy-update claim. Most of the newer cursors are checkpoints. Saying so in the API notes is treated as part of shipping, not as an admission of failure.

The Shared Contract

A contract that only one implementation upholds is a comment.

Before the structures, the rules they all obey. This chapter is short, dense, and load-bearing: nearly every specimen card later in the book is an application of something defined here.

2.1 Vocabulary

Term	Meaning in this library
Persistent value	An operation that looks like an update returns a new value or handle and preserves every retained old version.
One-way editing session	A single-owner edit-then-publish lifecycle. It is unsynchronized, consumed by publication, and is <i>not</i> a reusable staging builder.
Structural sharing	Versions reuse immutable substructure; no-op updates often preserve the same root or instance where the API exposes that diagnostic.
Reference-first equality	Two provably identical references may be accepted as equal without invoking user equality. A <i>non</i> -identical reference never proves inequality.
Policy preservation	Hash, equality, comparison, measure, allocator, ownership, and callback policies flow into derived versions unless an API explicitly creates a new policy.
Stable but unspecified order	Enumeration is deterministic for an unchanged version, but is not sorted, not insertion order, and not a serialization contract.

Insertion / explicit-position order	Enumeration follows first insertion plus unmistakable positional operations. Equality decides membership; it does not decide order.
Measure	A monoidal summary cached on a node, chunk, or facade element, used for split, locate, rank, priority, interval, and text navigation.
Tag action	A tag monoid acting consistently on elements and on cached ordered measures; composition order and distribution laws are part of the public policy contract.
Version-bound cursor	An immutable value owning one collection version and a position within it. Navigation and editing never silently redirect it to a different version.
Checkpoint port	A port that preserves observable semantics and tests while documenting a remaining representation or asymptotic parity boundary.
Facade	A public collection built on a shared core engine — sorted sets on measured trees, text ropes on measured ropes.

2.2 Policies are values, and they are retained

Almost every collection in this library carries a *policy*: an equality and hash pair, a comparison, a measure monoid, a tag algebra, a set of ownership callbacks, a codec, an allocator. The policy is not a construction-time convenience that gets forgotten — it is part of the collection’s identity and flows into every successor, *including empty ones*. Clear a map and the result is an empty map with the same comparer, not a default one.

Two rules follow that catch people out.

The receiver’s policy wins. When you union two sets, the result uses the receiver’s equality policy. The argument is normalized under that policy first — eagerly and completely, before any shortcut is applied — so that a policy or enumeration failure in the argument is visible rather than hidden behind an optimization that happened to skip it.

Policy identity gates structural shortcuts. Several structures can compare or merge in lockstep, interpreting stored hashes structurally instead of performing lookups, but only when the implementation has a positive witness that both operands retain *one* hash-policy identity. Two independently created but semantically compatible policies are not enough: their coherent hash states can differ. In that case the implementation falls back to lookup-based comparison, or rejects the operation explicitly, and says which it does.

2.3 Representatives

When two distinct objects are equal under the policy, exactly one of them ends up stored. Which one is not left to chance.

First representative, last payload

Construction and duplicate insertion retain the *first* stored key representative. Replacing the value for an equivalent key preserves the originally stored key object and installs the new value — so a map is “first key wins, last value wins”. Set algebra preserves receiver representatives for surviving receiver classes, and the first normalized argument representative for argument-only classes.

This sounds pedantic until the stored objects carry information that equality ignores — a cached parse, a source location, a case-preserved spelling under a case-insensitive comparer. Then it is the difference between a correct program and a mystery.

2.4 No-op identity

Operations that logically change nothing return the receiver. Removing an absent key, clearing an empty collection, adding an item a set already has, replacing a value with one that compares equal, applying an identity tag, seeking to the position you are already at: all of these hand back the same instance, and where a language has reference identity, the same reference.

This is more than an optimization. It is an observable, tested property that downstream code uses to short-circuit change propagation — a UI that re-renders when the model reference changes, an incremental compiler that skips a subtree whose table did not move. Because it is observable, the library treats it as contract and tests it, rather than as a happy accident of the implementation.

2.5 Failure atomicity

User code runs inside these structures: comparers, hash functions, measure policies, tag algebras, codecs, factory delegates, allocators. Any of it can fail.

Nothing partial is ever published

If a comparison throws, an allocation fails, a codec rejects input, or a factory raises, the operation publishes no successor. The receiver remains valid, every retained version remains valid, and no composite is left with one index updated and another not. Composites — bimap, relations, indexed maps, interval maps, graphs — publish *all* of their constituent indexes together or publish nothing at all.

Validation order is part of the contract too, and it runs in a deliberate sequence: argument validation precedes policy callbacks, range checks precede tag application, proof-shape limits precede allocation and hashing. The reason is that callbacks are the expensive and dangerous part; a malformed request should be rejected before any of it runs.

2.6 Ordering, stated honestly

Three different order contracts appear in this library, and conflating them is the most common way to write code that breaks on an unrelated change.

Contract	Who has it, and what you may rely on
Stable but unspecified	CHAMP maps, sets, bags, bimap, multimaps, relations. Deterministic for an unchanged version; <i>not</i> sorted, <i>not</i> insertion order, and never a serialization format. Tests exist specifically to stop consumers depending on the current trie order.
Insertion / explicit position	The neutral Ordered family. First insertion plus explicit moves, ranges, reversal, and one-shot stable sort. Equality decides membership, never position.
Comparer order	Sorted collections, canonical sets, priority search queues, interval structures, Patricia tries (ascending signed key order), Merkle trees (policy comparer order). Fully specified and safe to depend on.

2.7 Ownership, nine ways

The same semantic contract lands in nine memory models. This table is the translation key for the rest of the book.

Language	Ownership model, and what its docs must pin down
C#	GC references, immutable collection values, single-owner transient sessions. Exceptions, nullable/miss behavior, structural sharing, builder publication versus one-way transient consumption.
C	Opaque handles with explicit copy/destroy, callback policy tables, injected allocators. Status codes, allocation failure, retained versus borrowed values, alias-wide consumption, callback lifetime.
C++	RAII values over shared immutable nodes; move-only sessions. Move/copy cost, exception behavior including throwing policy moves, policy object lifetime.
Haskell	Pure immutable values plus explicitly scoped IO sessions. Total versus Maybe operations, session consumption, package-local classes, strictness.
Kotlin	Immutable JVM values, runtime policies, dynamically consumed sessions. Null/result/exception shape, version-bound views, comparator behavior.
OCaml	Immutable algebraic values, runtime policy records, dynamically consumed sessions. <code>option/result</code> boundaries, policy identity, Unicode-scalar text positions, mutex-coordinated facades.
Rust	Owned values, borrows, Arc sharing, consuming sessions. Clone requirements, borrowed lookup results, panic boundaries, <code>Send/Sync</code> claims only when proven.
TypeScript	GC objects, immutable versions, runtime policies, dynamically consumed sessions. ESM exports, undefined/miss shapes, stable hashing versus JavaScript identity, isolate-local concurrency, UTF-16 indexing.
Python	GC objects, immutable versions, retained policies, dynamically consumed sessions. Typing and runtime validation, <code>None</code> versus absence, equality/hash coherence, iterator invalidation, code-point indexing.

Presence versus null

Every port has to distinguish “the key is absent” from “the key is present and its value is null-like”. The library never collapses the two. C# uses `TryGetValue` out-parameters; Rust and OCaml use `Option`; Haskell uses an outer `Maybe` (so a stored `Nothing` at element type `Maybe a` comes back as `Just Nothing`); Kotlin, TypeScript, and Python use explicit presence-bearing result wrappers rather than a bare nullable return. The authenticated three-way merge takes this furthest: it distinguishes a deleted key from a key whose merged value is present-and-null, because on the wire those are different facts.

PART II

The Unordered World

CHAMP: The Hash Trie at the Center

Every third structure in this library is a hash trie wearing a different hat.

Specimen — persistent hash map and set

Purpose	Unordered key/value and membership collections with structural sharing, policy-defined equality, and structural equality/diff.
Core	CHAMP: a 32-way bitmap-indexed prefix tree with split data/node bitmaps and canonical deletion topology.
Lookup	$O(\log_{32} n)$ — at most seven levels for 32-bit hashes, plus bucket work on a full-hash collision.
Update	$O(\log_{32} n)$ path copy.
Equality/diff	Lockstep slot alignment with reference pruning where policy identity is witnessed; $O(n + m)$ for independently built operands.
Order	Stable but unspecified. Never sorted, never insertion order.
Ports	PersistentHashMap/PersistentHashSet (C#, Kotlin, Rust, TypeScript, Python), persistent_hash_map/_set (C++), HashMap/HashSet (Haskell), d7_hamt_map/d7_hamt_set (C), Persistent_hamt / Persistent_hash_set (OCaml).

Start here, because a surprising amount of the library is downstream of this one structure. The hash bag, the bidirectional map, the set-valued multimap, the relation, the map patch, the directed graph, the indexed map, and the insertion-ordered set are all — in whole or in part — compositions over the persistent hash map. Understand its representation and you have understood the substrate for a third of the collections in

this book.

3.1 From HAMT to CHAMP

Bagwell’s hash array mapped trie is the classic design: consume five bits of the hash per level, use a 32-bit bitmap to record which of the 32 slots are occupied, and store only the occupied slots in a compact array. Population count on the bitmap below the slot’s bit gives the array index. Lookup is a handful of hops; the tree is shallow because $32^7 > 2^{32}$.

CHAMP — the Compressed Hash-Array Mapped Prefix-tree, from Steindorfer and Vinju’s OOPSLA 2015 paper — keeps that skeleton and changes two things.

3.1.1 The bitmap split

A CHAMP node keeps *two* bitmaps: a datamap recording which slots hold an inline key/value payload, and a nodemap recording which slots hold a pointer to a sub-node. The single backing array is then divided into two regions: payloads packed at the front, sub-node references packed at the back (Figure 3.1).

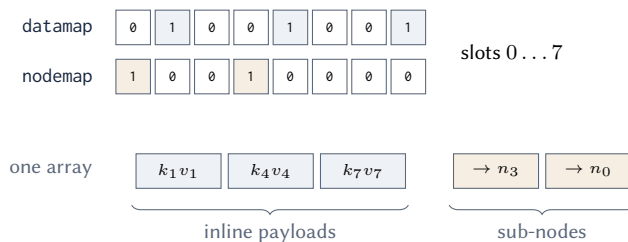


Figure 3.1: A CHAMP node. Two bitmaps classify each of the 32 logical slots as empty, inline payload, or sub-node. The payloads live contiguously at the front of one array and the sub-node references at the back, so iteration over entries never has to step past pointers.

The immediate win is layout. Iteration touches payload runs contiguously instead of interleaving pointers and payloads, and a lookup that lands on a data slot reads the entry directly rather than dereferencing a leaf object. The deeper win is the second change.

3.1.2 Canonical deletion topology

In a plain HAMT, deletion leaves scar tissue. A sub-node that shrinks to a single entry stays a sub-node, so the tree’s shape encodes its update history: two maps with identical

contents can have different skeletons depending on what was inserted and removed along the way.

CHAMP normalizes on the way out. When a sub-node shrinks to a single payload, it is collapsed into its parent’s data region; when a wrapper node ends up holding nothing but a single child, the wrapper is removed (Figure 3.2). Bitmap topology is therefore determined by the hashes that remain, not by the order in which they arrived.

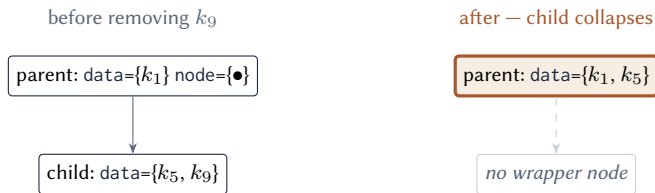


Figure 3.2: Canonical deletion. A sub-node that would be left holding one payload is promoted into its parent, so shape depends on contents rather than on history.

That canonicalization is what makes the interesting operations possible.

3.2 Equality and diff without a lookup pass

If two maps with equal contents have the same skeleton, then comparing them does not require looking every key of one up in the other. You can walk both trees in lockstep, aligning logical bitmap slots, and compare structure directly. Better: when two versions share ancestry — which they will, constantly, in a persistent library — entire subtrees are *reference-identical*, and a reference comparison prunes them without touching a single key.

Structural equality and diff

Where the implementation has a positive witness that both operands retain one hash-policy identity, `MapEquals` and `Diff` traverse aligned logical slots using stored hashes and prune every reference-identical descendant. Work then tracks the visited non-shared regions plus the reported differences. Independently created but compatible policies fall back to lookup-based comparison, because their coherent hash states may differ.

Identical topology is not reference identity

Two maps built independently — same entries, same policy, different allocations — share nothing. Comparing them is an honest $O(n + m)$ traversal, and CHAMP by itself does *not* guarantee work proportional to logical divergence. The benchmark suite deliberately separates “shared ancestry, one change” from “independent histories, equal contents”, because those two cost profiles are different and a single number would hide it.

Collision buckets add their own asymmetry. Their internal order is insertion order, so equality has to match keys as an unordered set, costing up to $O(c^2)$ comparisons for a bucket of size c . Equal maps may also retain different concrete key objects, because “first representative wins” is per-history.

Structural algebra — union, intersection, difference over two same-type maps or sets — uses the same machinery: align slots, prune shared descendants, and build the result bottom-up.

3.3 Collisions

Two keys whose *entire* 32-bit hash is equal cannot be separated by any number of levels. CHAMP handles this with an immutable collision bucket: a small array of entries that share a hash, searched linearly under the equality policy. This is strictly better than the alternative of extending the trie past the hash — there is nothing left to extend it with — and it degrades gracefully, because a pathological hash function turns the map into an association list rather than into an infinite loop.

Unary bitmap chains are a related and often-misunderstood shape. Even under canonical deletion, a node can legitimately have exactly one child: two keys whose hashes agree on the first several five-bit groups must be separated deeper down, and the intervening nodes have nothing else to hold. Canonical here means *canonical topology given the hashes present*, not “no node ever has one child”.

3.4 Single-descent factories

Two operations recur so often in real code that all nine ports expose them directly rather than making callers compose lookup with update:

```
| var withThree = map.GetOrAdd(
```

```

3,
key => $"value-{key}",
out string threeValue);

var counted = PersistentHashMap<string, int>.Empty.AddOrUpdate(
    "requests",
    _ => 1,                                // absent: seed
    (_, current) => checked(current + 1),    // present: update
    out int requestCount);

```

Both hash once and descend once. That is the point: the naive composition (TryGetValue then SetItem) hashes twice and walks the trie twice. The contract around them is tight, and worth stating in full because it is easy to get subtly wrong.

One descent, one factory, no retry

Delegates are validated *before* hashing. Exactly one factory is selected and invoked exactly once; GetOrAdd invokes none on a hit. There is no retry loop — unlike a mutable concurrent map, a persistent update has no competing writer to lose a race to. An update factory receives the caller’s lookup key and the stored value, while the map retains the originally stored key object. If the produced value compares equal to the stored one, the source map and the stored value object are both retained. Any factory or comparer exception leaves the source unchanged and publishes nothing.

3.5 Sets, and what changes

PersistentHashSet is the same trie with payloads that carry no value. Its distinctive surface is set algebra and the six set relations — subset, proper subset, superset, proper superset, overlap, and set equality — all of which preserve the receiver’s policy and normalize the argument under it first.

Set algebra benefits from the same reference pruning as map equality: unioning a set with a descendant of itself skips every shared subtree. Unioning two unrelated sets does not, and the API notes say so.

3.6 Nine ports, one representation

Every port ships canonical split-bitmap CHAMP nodes, collision buckets, persistent path copying, retained hash policies, and stored representatives. Where they differ is in the optimization tier layered on top:

Port	Notes beyond the shared representation
C#	The reference implementation, and the only one with the owner-token transient kernel (Chapter 7). Lockstep reference-pruned MapEquals, bitmap-aligned structural Diff.
Kotlin	Same split bitmaps and canonical promotion; exact-policy mapEquals and typed diff traverse logical slots in lockstep using stored hashes.
Rust	Safe Arc nodes, <code>#![forbid(unsafe_code)]</code> . Both path copying and BulkBuilder freezing produce canonical nodes. Independently created hasher policies keep semantic lookup fallback.
Haskell	Strict split data/node maps, inline <code>(hash, key, value)</code> runs, typed MapDifference values, and indirection-aware pointer pruning. validStructure checks the CHAMP invariants.
C++	Header-first C++20, move-only bulk builder, compact inline payload vectors. Independently created stateful hash policies retain semantic equality/diff.
C	C17, split maps, inline type-erased payloads, flexible-array child runs, retain/release policy balance, allocation-failure rollback. Diff is visitor-based so it does not impose an allocator on callers.
TypeScript	Canonical CHAMP with reusable public bulk builders; retains the structural reference-pruning checkpoint.
Python	Canonical CHAMP with the same public semantics, but deliberately performs semantic lookup-based equality and diff (with exact-policy fast paths) rather than claiming the lockstep descendant-pruning optimization.
OCaml	Immutable 32-way CHAMP with the same public semantics; descendant-pruning is implementation-specific and not claimed.

Why enumeration order is a tested non-promise

It would be easy, and wrong, to let consumers depend on the current trie order. Durable7 has tests whose entire purpose is to stop that: they assert that independently built equal maps may enumerate differently, and they exist so that a

future representation change — a size-tiered small layout, say — cannot break downstream code that quietly assumed otherwise. The one place a hash trie *does* promise an order is Ctrie snapshot conversion (Chapter 6), which fixes canonical data-run-before-node-run order precisely so that snapshot-to-persistent conversion preserves the captured sequence.

Companions of the Trie

The cheapest new data structure is two old ones that agree with each other.

Nine families in this library are built by composing existing substrates rather than by introducing a new core tree. That is a deliberate rule, not an accident of laziness: a new core is a new set of invariants, a new validator, a new adversarial test suite, and nine new implementations. A composition inherits all of that from its parts, and the only genuinely new obligation is keeping the parts consistent with each other.

The composite rule

A composite publishes *all* of its constituent indexes together or publishes nothing. Counts agree across indexes; no entry exists in only one of them. Policy-bearing composites retain independent key/value, left/right, or equality/ordering policies as appropriate, and every mutation preserves retained versions and returns the receiver for documented logical no-ops.

4.1 Persistent hash bag

Specimen — PersistentHashBag

Purpose	Unordered multiset: one representative and one positive multiplicity per equality class.
Built from	One CHAMP map from representative to count.

Counts	Distinct-class count and expanded total count are separate values. No ambiguous single “count” is exposed.
Algebra	Maximum union, minimum intersection, saturated difference, checked additive sum.
Ports	All nine.

The bag stores multiplicities in $1 \dots 2^{31} - 1$ per class — never zero, because a zero-multiplicity class is an absent class. The expanded total is carried in a wider checked integer, and the choice of type is a small tour of the nine runtimes: C# uses a checked long, TypeScript a `bigint`, Python an unbounded `int`, OCaml a checked native `int`, and the native ports their corresponding bounded wide integer.

Bag algebra is where the receiver-policy rule earns its keep. A reference-different argument comparer is normalized eagerly under the receiver’s policy *before* any operation-specific shortcut, and collapsed argument multiplicities are checked for overflow and use the first representative observed in that argument version’s trie order. Expanded enumeration repeats each representative contiguously; distinct-item and entry views enumerate one class each, in the usual stable-but-unspecified order.

4.2 Persistent bidirectional map

Specimen — PersistentBiMap

Purpose	A strict bijection: each key class and each value class appears at most once.
Built from	A forward CHAMP $K \rightarrow V$ and an inverse CHAMP $V \rightarrow K$, with independently retained policies.
Inverse	$O(1)$ in pair count; enumerates nothing.
Storage	Approximately two map entries per logical pair — stated plainly rather than hidden.
Ports	All nine.

The bimap is the clearest illustration of the composite rule, because its consistency obligation is symmetric and total: both maps have the same count, every forward

entry has exactly one equivalent inverse entry, and every inverse entry has exactly one forward entry. There is no operation that can be halfway done.

The interesting decisions are in the update semantics.

- ▶ **Strict add rejects both ways.** Adding a pair whose key class is occupied fails; so does adding one whose value class is occupied — even when the complete pair is equivalent to one already present. The key domain is checked first, so a result-bearing API reports key conflict before value conflict. (C# deliberately keeps its established boolean `TryAdd` shape rather than growing a richer result.)
- ▶ **Set is not force-put.** Setting adds a missing free pair; treats a policy-equivalent value update as a no-op that retains both representatives; replaces a present key's value *only* when the new value class is free; and never displaces another key. There is no displacing mode, deliberately.
- ▶ **Replacement goes the long way round.** It removes and reinserts both directions rather than using the ordinary map's value-equality shortcut, because that shortcut consults a different policy than the one that governs the inverse domain.
- ▶ **Double inversion is free and exact.** C#, Kotlin, TypeScript, and Python cache reciprocal facade objects, so `map.Inverse.Inverse` is the original object. The value-semantic ports (C, C++, Haskell, Rust) guarantee the analogue: double inversion shares the original two immutable roots.

The family intentionally exposes no algebra, no builder, no transient, and no force-put. Each of those would need a bijection-preserving story, and none has a consumer asking for it.

4.3 Set-valued multimap and relation

Specimen — `PersistentHashMapMultimap` and `PersistentRelation`

Purpose	One key to many values (multimap); many-to-many with cheap inverse traversal (relation).
Built from	Multimap: CHAMP map from key to non-empty CHAMP set. Relation: two multimaps, forward and reverse.
Counts	Distinct-key count and checked pair count are separate.

Inverse

Inverse swaps the relation's two persistent indexes without scanning the pair set.

Ports

All nine.

A multimap stores *only* non-empty value sets: removing the last pair for a key removes the key. That single invariant is what makes “does this key have any values” a membership test rather than a set-emptiness test, and it is why duplicate pair insertion and removal of an absent pair are identity no-ops.

The relation adds the reverse index, so asking “which lefts point at this right” is a lookup rather than a scan. Adding or removing a pair updates both directions atomically; on failure, both indexes and every retained version remain mutually consistent.

4.4 Map patch

Specimen — PersistentMapPatch

Purpose

A first-class, invertible, composable description of a change to a map.

Semantics

Distinguishes absence from every present value, including null-like ones.

Apply

Validates *all* before-states before applying *any* edit; preserves the source map on conflict.

Invert

Swaps before- and after-states.

Compose

Only through equal intermediate states.

Ports

All nine.

The patch is the library's answer to “I want to describe a change, inspect it, send it somewhere, and maybe undo it”. Its preflight discipline is the whole design: a patch either applies completely or does not apply at all, and it will refuse rather than apply half of itself to a map whose before-state has drifted. The absence-versus-null distinction matters here more than almost anywhere else, because “set this key to null” and “delete this key” are different patches that a sloppier encoding would conflate.

4.5 Directed graph

Specimen — PersistentDirectedGraph

Purpose	Explicit vertices and unique directed edges with forward and reverse adjacency.
Built from	A vertex set plus two adjacency multimaps.
Vertices	Stored explicitly, so an isolated vertex is a real vertex. Adding an edge adds any missing endpoints.
Removal	Removing a vertex removes every incident edge.
Ports	All nine.

Storing vertices separately from edges is the design decision worth noticing. An edge-list-only graph cannot represent an isolated vertex, which means it cannot represent “I have declared this node but not yet connected it” — a state that occurs constantly in dependency graphs and build systems.

4.6 Indexed map

Specimen — PersistentIndexedMap

Purpose	A primary map with one automatically maintained non-unique secondary index.
Built from	The primary CHAMP map plus a secondary multimap keyed by a selector result.
Invariant	Exactly one secondary pair per primary key; changing a row’s selector class moves its membership atomically.
Ports	All nine.

This is the “find all users in this city” structure. The selector is applied on insertion, and the composite’s job is to keep the derived index honest across every edit — including the awkward one, where updating a row changes which secondary bucket it belongs to.

4.7 Persistent chunked bit set

Specimen — PersistentChunkedBitSet

Purpose	A sparse set of non-negative 32-bit integers with population rank and select.
Core	64-bit words at the leaves of a measured tree, with a popcount sum measure.
Membership	$O(\log c)$ for c represented chunks.
Rank/select	Inclusive rank and zero-based select by measured descent.
Algebra	Word-wise merge over sparse streams — never a scan to the largest index.
Ports	All nine.

The chunked bit set is filed with the trie companions because it plays the same role — a set — but its representation belongs to the sequence world: 64-bit words at the leaves of a measured tree whose measure is the population count of its subtree. That measure is what turns Rank and Select into descents rather than scans, and it is a small, satisfying demonstration of how much the measure framework (Chapter 9) buys.

Zero words are never stored, so storage is proportional to represented chunks rather than to the largest bit index, and set algebra walks two sparse word streams rather than iterating a dense range.

Which set do you actually want?

Three families in this library can represent “a set of integers”, and they are good at different things. `PersistentHashSet` is the general answer with no ordering. `PersistentIntSet` (Chapter 5) gives ascending signed order and *structural* merge. `PersistentChunkedBitSet` gives rank and select, and wins hardest when the integers are dense within clusters and the workload is set algebra. Reach for the third only when rank/select or word-wise algebra is the reason you are here.

Patricia: When the Key Is Its Own Hash

A hash trie hashes the key. If the key is already a number, that step is pure ceremony.

Specimen — Patricia integer maps and sets

Purpose	Persistent maps and sets over signed 32- or 64-bit integer keys, with ascending key order and structural merge.
Core	Big-endian path-compressed binary trie: each internal node stores a common prefix and the single bit on which its two subtrees differ.
Point ops	$O(\min(n, W))$ for key width W — in practice a few node hops.
Algebra	Prefix-aligned union, intersection, difference, with combining-function overloads. Work follows visited non-shared prefix regions; $O(n + m)$ worst case.
Order	Ascending <i>signed</i> order, via a sign-bit transform.
Ports	All nine, at both widths: PersistentIntMap/Set and PersistentLongMap/Set and their local spellings.

The hash trie is a fine general answer, and integer keys are the case where it is least differentiated: hashing an `int` to route it through a trie is a detour, because the key already *is* a well-distributed bit string. Okasaki and Gill’s 1998 paper on mergeable integer maps supplies the structure that takes the shortcut, and Haskell’s `Data.IntMap` has been the proof of its practicality for two decades.

5.1 Path compression and the branching bit

A binary trie over the bits of an integer key would be W levels deep and mostly empty. Patricia collapses the empty stretches: each internal node stores the prefix its subtree shares, together with the single bit position at which its two children diverge. Descent is then a prefix test and a bit test — no comparisons, no rebalancing, and no rotations, ever.

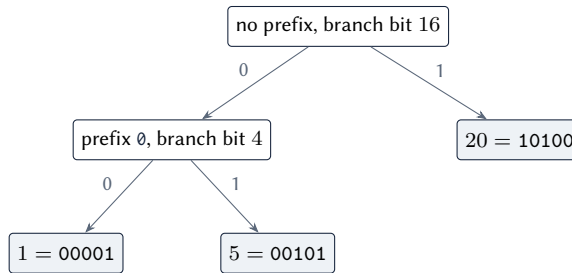


Figure 5.1: A Patricia trie holding $\{1, 5, 20\}$. Keys 1 and 5 agree on every bit above position 4, so the intervening levels are compressed away and the node records only the prefix and the branching bit.

Two pleasant properties fall out immediately. There are no collision buckets, because distinct keys are distinct bit strings — the structure is strictly simpler than the hash trie. And enumeration is in unsigned key order for free, since the trie is ordered by construction.

Signed order, by transform

Raw trie order is unsigned, which puts -1 after $2^{31} - 1$. Every port applies a sign-bit transform so that public enumeration is in ascending *signed* order, and every port ships both an explicit 32-bit and an explicit 64-bit surface rather than a platform-sized one. Python holds arbitrary-precision integers at runtime but validates and normalizes keys to the named fixed-width domain at the public boundary; Haskell exposes `Int32/Int64` aliases so the width contract does not depend on a platform `Int`.

5.2 The reason it exists: structural merge

Point operations are not the interesting part. The differentiator — the thing the hash trie genuinely cannot match — is *structural merge*.

Union, intersection, and difference align compressed prefixes and descend both tries together. Where the two prefixes are disjoint, the answer is known without looking inside either subtree. Where the two subtrees are reference-equal, the answer is one of them, and the recursion short-circuits. Work is proportional to the visited non-shared prefix regions plus the materialized result, and a no-op merge preserves reference identity outright.

That is a categorically different cost profile from merging two hash maps, which must enumerate one and look up each key in the other. Merge-heavy workloads — version-set algebra, sparse indices, identifier-set reconciliation, permission masks — stop paying a per-key hash-and-descend and start paying only for where the two operands actually differ.

```
var baseline = PersistentIntMap<string>.CreateRange(
    new[] { KeyValuePair.Create(-1, "old"), KeyValuePair.Create(10, "ten") });
var delta = PersistentIntMap<string>.CreateRange(
    new[] { KeyValuePair.Create(-1, "new"), KeyValuePair.Create(20, "twenty") });

var merged    = baseline.Union(delta);                // right-biased
var combined = baseline.Union(delta,
    (key, left, right) => $" {key}:{left}+{right}");

foreach (var key in merged.Keys)
    Console.WriteLine(key);    // -1, 10, 20 -- ascending signed order
```

The combining-function overloads are what make structural merge *useful* rather than merely fast. Without them, a union has to pick a side and discard information; with them, a merge can reconcile. The vocabulary is deliberately modeled on `Data.IntMap`'s `unionWith/intersectionWith`, because that is the shape that has proven itself in practice.

Sharing lowers the constant, not the bound

Retained sharing reduces the number of visited regions; it does not define a separate asymptotic class. The honest worst case remains $O(n + m)$, and two independently constructed tries with identical contents will be walked in full.

As always in this library, the good case is documented as a good case rather than promoted to a promise.

Ctrie: Snapshots Without Stopping the World

A mutable map you can photograph in constant time.

Specimen — ConcurrentHashTrie

Purpose	A lock-free <i>mutable</i> concurrent hash map whose <code>Snapshot()</code> is $O(1)$ and yields an immutable view.
Core	Bitmap C-nodes, singleton leaves, equal-hash collision nodes, and tomb nodes, with generation-stamped indirection nodes.
Mechanism	Node-local GCAS for safe indirection replacement; root/main RDCSS for the snapshot transition; lazy generation renewal on later writes; tomb promotion on deletion.
Snapshot order	Canonical CHAMP order — data run before node run at every level — so conversion preserves the captured sequence exactly.
Ports	Full lock-free protocol: C# and Kotlin/JVM. Synchronous or lock-coordinated snapshot facades: OCaml, TypeScript, Python. Deliberately absent elsewhere.

Every other structure in this book is immutable. This one is not, and it earns its place by solving a problem the immutable structures create.

If your workload is a hot shared cache — hundreds of thousands of writes a second from many threads — a persistent map is the wrong tool, because every write allocates a path and every writer contends on publishing a new root. But if that same cache periodically has to hand a *consistent view* to a reporting thread, a checkpointer, or a

serializer, then a plain concurrent hash map is also the wrong tool, because taking a consistent view means locking everybody out or copying everything.

Prokopec, Bronson, Bagwell, and Odersky’s concurrent trie (PPoPP 2012) sits exactly in that gap: mutable-map throughput with $O(1)$ snapshots.

6.1 How a constant-time snapshot is possible

The trick is generation stamping. Every indirection node carries the generation in which it was created. `Snapshot()` does not copy anything; it atomically installs a new root with a bumped generation. From that moment, any writer that wants to modify a node belonging to an older generation must first *renew* it — copy it into the current generation — before writing. Readers holding the old root keep seeing old-generation nodes, which nobody will ever mutate again.

The subtlety, and the reason this algorithm is famous, is that “atomically install a new root” has to compose with “atomically replace a node” without losing a writer in between. Two mechanisms cooperate:

GCAS — generation-compare-and-set — makes an indirection node’s replacement safe by publishing a descriptor that readers help complete. A reader that finds a node mid-transition finishes the transition rather than waiting for it.

RDCSS — restricted double-compare single-swap — makes the root/main transition atomic with respect to the main-node observation, so a writer cannot slip between “I read the main node” and “the root was replaced”. The C# implementation gives GCAS priority in that transition specifically to close that window.

Deletion adds a third concern. Removing entries from a deep trie would otherwise leave a skeleton of empty interior nodes that future traversals still have to walk. The Ctrie contracts these paths by promoting *tomb* nodes — empty or singleton markers — up through their parents, so the structure does not accumulate scar tissue.

6.2 Crossing back into the persistent world

A snapshot is only useful if you can hand it to code that expects an immutable collection. Conversion is explicit and $O(n)$, and its contract is unusually specific:

Snapshot conversion preserves the captured sequence

Snapshot enumeration follows canonical CHAMP ordering: at each bitmap level, singleton and data entries precede multi-entry child runs; frozen empty tombs are skipped; frozen singleton tombs are promoted logically into the data run; and equal-hash collision buckets retain their local order. Explicit snapshot-to-`PersistentHashMap` conversion then preserves that exact sequence, the exact policy object, stored key and value representatives, present-null semantics, and isolation from every later live write.

This is the one place in the library where a hash structure commits to an enumeration order, and the reason is precisely that conversion must be *exact* rather than merely correct.

6.3 Why only two ports have the real thing

The reclamation problem

Lock-free node replacement is easy to write and hard to free. A thread may be holding a pointer into a node you have just unlinked, and nothing in the algorithm tells you when the last such thread is done.

On the CLR and the JVM the garbage collector solves this for free, which is why C# and Kotlin ship the complete protocol. In C and C++ it would need epoch-based reclamation or hazard pointers — a separate project with its own correctness burden. Rust's `Arc` does not help either: sharing an immutable graph is not the same problem as atomically swapping child pointers, and a safe design would mean an `ArcSwap` or epoch dependency and a new core. Haskell has a collector but no use for an imperative GCAS algorithm behind a pure API; STM or an atomic root wrapper would be a different structure wearing the same name. The parity economics therefore cap the lock-free tier at two managed ports, and the porting guide says so out loud rather than leaving a hole in a matrix.

OCaml, TypeScript, and Python ship something honest but different: a facade with the same consumer-level vocabulary — a mutable map, $O(1)$ immutable snapshots — implemented by serializing operations (an `RLock` in Python, a mutex in OCaml, isolate-locality in TypeScript) and swapping immutable CHAMP roots. The names preserve

the *use case*. They do not claim GCAS, RDCSS, generation renewal, tomb cleanup, or a lock-free progress guarantee, and their API notes say so in the first paragraph.

Builders, Sessions, and the Frozen Tier

Immutability is a promise about what others can observe, not a vow of celibacy toward your own unpublished nodes.

Building a million-entry persistent map one `SetItem` at a time allocates a million paths, and only the last one survives. Everyone who has built a persistent collection library has arrived at the same conclusion: there must be a way to mutate structure that nobody can see yet.

Durable7 keeps *five* distinct concepts here, and refuses to blur them, because each has a different lifecycle and a different set of promises (Figure 7.1).

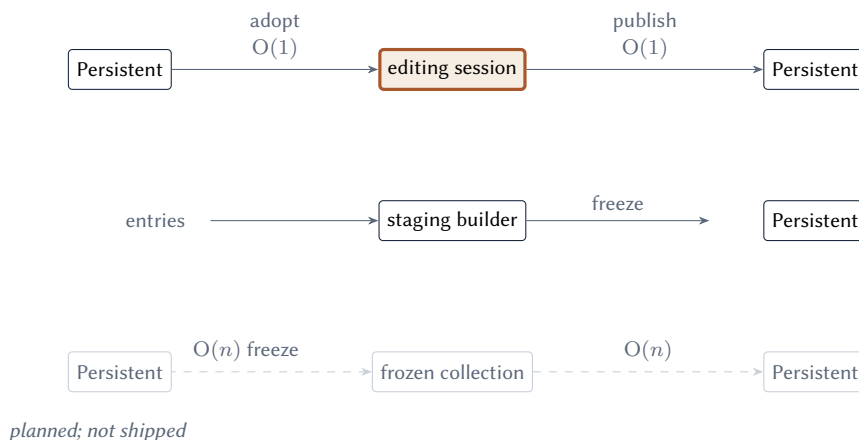


Figure 7.1: Five distinct lifecycles, kept deliberately separate. The session is one-way and consumed by publication; the staging builder is reusable and produces detached snapshots; the frozen tier remains an evidence-gated proposal.

7.1 Construction-only bulk builders

A bulk builder mutates unpublished leaf, collision, and bitmap nodes in place during a single construction pass, avoiding a path copy between successive input entries. It is narrower than an editing session and its contract says so.

Bulk builder

Nodes under construction are unpublished and mutated in place. Duplicate assignment retains the *first* stored key representative, keeps an earlier equal value representative, and otherwise installs the last distinct value. Freezing produces a *detached* immutable trie — later builder mutation cannot affect an earlier snapshot — and a reusable freeze leaves the builder active. Map and set bulk factories use the builder internally; ordinary updates on an existing persistent collection continue to use structural-sharing path copies.

C# uses one internally; C++, Rust, TypeScript, and Python expose the facility publicly. Rust additionally offers a consuming freeze that moves owned nodes rather than sharing them. OCaml’s reusable staging builder preserves the detached-freeze semantics and the representative rules but performs persistent path copying internally, and explicitly does not claim the unpublished mutable-node construction bound.

7.2 The one-way editing session

A session is the answer to “I have an existing map and I want to make eighty changes to it”. It adopts a persistent root in $O(1)$, accepts edits under single ownership, and publishes once.

One-way lifecycle

Adoption and successful publication perform $O(1)$ trie work and do not traverse the graph. Sessions preserve the exact equality/hash policy, stored representatives, unchanged-version trie order, retained source isolation, and receiver-policy set-relation semantics.

Publication is *one-way*: it consumes the logical session, and the language either rejects later access dynamically or makes it unavailable through ownership. A session is unsynchronized and has one logical owner, even where the language

permits explicit handle aliases or moves.

Logical no-ops preserve the current root and do not invalidate a version-bound view. A clean adopted session therefore republishes the exact source instance. A failed edit never installs a partial successor.

7.2.1 One optimized kernel, eight semantic siblings

This is the sharpest example in the library of the reference-versus-checkpoint distinction, and it is worth being precise about.

C# implements the *owner-token transient*: edits mutate only nodes and arrays proven to be owned by the active edit token, and path-copy anything shared or sealed. `Persist()` seals the token, publishes without traversing the graph, and invalidates the mutable object along with every view and enumerator obtained from it. A clean `source.ToTransient().Persist()` returns the exact source object, including after logical no-op edits.

The other eight ports implement the same *observable lifecycle* over ordinary persistent path copying. Each changed point edit computes a normal path-copied successor and replaces the session's current value. They preserve policy identity, stored representatives, exact clean/no-op root identity, retained sources, and failure atomicity — and they make *no* transient-throughput or allocation-win claim. That is stated in their API notes, not discovered in a profiler.

Port	Lifecycle and failure shape
C	Explicit clone/destroy handles over one reference-counted state. Clones are lifecycle <i>aliases</i> : one successful publication consumes them all. Operations report a consumed or modified status; failed edits and relation queries preserve session and output atomicity.
C++	Sessions are movable but not copyable and publish only through <code>std::move(session).persist()</code> . Generation-bound iterators reject changed or consumed sessions. A throwing custom policy move terminally invalidates the source (and, for assignment, the destination), and publication after such a move carries no retry guarantee — nothrow-movable policies avoid both boundaries.
Haskell	Sessions live in IO and are consumed by <code>persistMap/persistSet</code> . Candidate construction precedes a masked commit, so neither synchronous nor asynchronous failure can partially install an edit.

Kotlin	Consumption is enforced at runtime. Acquired views capture the current snapshot and session version, survive logical no-ops, and fail after content changes. Callback failure leaves the session unchanged and retryable.
OCaml	The session retains one persistent root, replaces it with path-copy successors, and dynamically rejects access after publication. The reusable bulk builder is a separate facility.
Rust	Publication is <code>into_persistent(self)</code> , so the type system prevents use-after-publication. <code>into_transient</code> moves a source; <code>to_transient</code> shares its root.
TypeScript	One-way publication enforced at runtime inside one JavaScript isolate; no cross-worker progress claim.
Python	Consumes on publication and invalidates iterators after content changes or publication.

7.3 The frozen tier, and why it has not shipped

The third lifecycle — $O(n)$ conversion to a read-optimized layout with no update-shaped API — is designed, prototyped, and deliberately unshipped.

Its intended contract is already written down: separate `FrozenHashMap` and `FrozenHashSet` types, one offline-built general hash layout for every count and key type, preserved comparer identity and stored representatives, preserved source traversal sequence, no library-controlled allocation on point lookup, and enumeration that is stable for an unchanged instance but otherwise unspecified — never insertion, sorted, canonical, or serialization order. Freezing an already-frozen value is identity; converting back is visibly $O(n)$.

What is missing is evidence. The track is staged: a faithful packed-index signal first, fixed layouts compared only once a credible read-heavy regime appears, and a public type authorized only behind lookup, enumeration, memory, and realistic construction break-even numbers. The benchmark-local candidates already cover the low-risk half — rebuilding canonical CHAMP from a packed source-order entry sequence through the existing bulk builder, with an untimed verifier locking comparer identity, traversal sequence, representative rules, null and collision behavior, and round-trip reconstruction.

Deliberate non-adaptivity

Tiny flat layouts, string prefix and length dispatch, integer-key routing, binary-fuse and ribbon filters, Eytzinger and PGM sorted layouts, and count-derived representation selectors are all postponed — not rejected, postponed. The stated reason is methodological: the value of the *temporal* lifecycle (transient, persistent, frozen) is being isolated before it is combined with size or key specialization, so that a later benchmark can tell which of the two produced a gain. Benchmarks may contain tiny and string datasets; that is a workload, not permission to select a representation.

PART III

Sequences

Finger Trees

A tree that keeps its hands in its pockets: whatever else is deep, both ends are always within reach.

Specimen — `FingerTreeDeque` and `FingerTree`

Purpose	A persistent indexed sequence with cheap endpoints and cheap concatenation (<code>FingerTreeDeque</code>), and a general monoid-measured tree used directly and as the engine beneath a dozen facades (<code>FingerTree</code>).
Core	Hinze and Paterson’s 2-3 finger tree: digits at each end, a recursively nested spine of nodes in the middle.
Endpoints	$O(1)$ amortized push and pop at either end.
Concat	$O(\log \min(n, m))$ — proportional to the <i>smaller</i> operand.
Index / split	$O(\log d)$, where d is the distance from the nearer end. The measured U-shape.
Ports	All nine: <code>FingerTreeDeque</code> / <code>PersistentDeque</code> / <code>persistent_deque</code> / <code>Deque</code> , plus the measured tree under each language’s local name.

Most balanced trees treat all positions alike. A finger tree does not: it keeps a small unbalanced “digit” of one to four elements at each end and pushes the balanced structure into the middle, where a recursively nested spine holds *nodes* of two or three elements, which themselves hold nodes, and so on (Figure 8.1).

That asymmetry is the whole design. Pushing onto an end usually just grows a

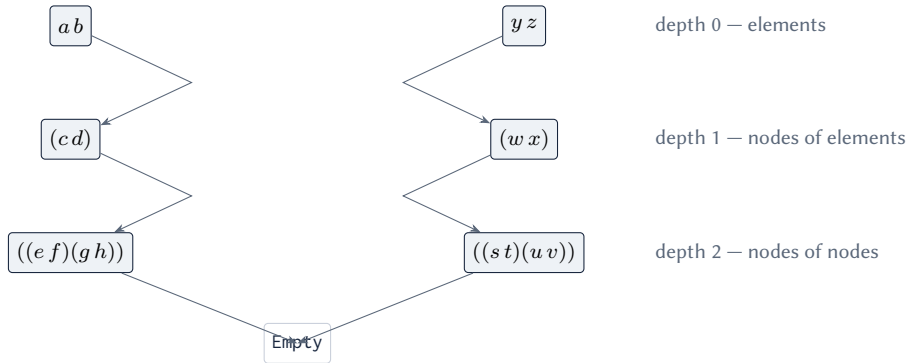


Figure 8.1: The finger-tree shape. Both ends sit at depth zero and stay there; only the middle gets deeper. This is why endpoint operations are constant-amortized while indexing costs a logarithm of the distance to the nearer end.

digit, which is $O(1)$; occasionally a digit overflows and a node is pushed one level down, and the amortized analysis — with memoized lazy suspensions spreading the debt across later forces — keeps the average constant. Concatenation is proportional to the logarithm of the *smaller* operand, because the two spines only have to be zipped together as deep as the shorter one goes. Indexing costs a logarithm of the distance to the nearer end, so both `seq[0]` and `seq[n-1]` are nearly free while the middle is logarithmic — the “measured U-shape” the API notes refer to.

8.1 The deque

`FingerTreeDeque<T>` is the tuned sequence facade. It exposes endpoint operations, concatenation, splitting, positional edits, and range operations.

```
var deque    = FingerTreeDeque<int>.Create(1, 2, 3);
var extended = deque.AddFirst(0).AddLast(4);

var first    = extended.PopFirst();
var rest     = first.Rest;           // [1, 2, 3, 4]

var split    = rest.SplitAt(2);
var joined   = split.Left.Concat(split.Right);
var replaced = joined.SetItem(1, 20); // [1, 20, 3, 4]
```

It also carries a small set of helpers for sequences the caller *knows* to be sorted

— `SortedLowerBound`, `SortedUpperBound`, `InsertSorted`, `SplitAtSortedEqualRange`. These assume sortedness rather than maintaining it, which is exactly the right split of responsibilities: if sortedness is a long-lived invariant, the sorted collections of Chapter 15 maintain it for you; if it is a passing property of a sequence you happen to have, these helpers exploit it without committing you to a different type.

8.2 The measured tree

The general `FingerTree<TElement, TMeasure, TMeasureOps>` is the deque’s more powerful sibling and the engine underneath sorted bags, sets and maps, priority queues, interval trees, ropes, and the chunked bit set. Every node caches the monoidal summary of its subtree, and the tree can be split at the first position where a predicate over the running prefix measure flips from false to true.

That single operation — *split by measure* — is startlingly general. Measure by element count and you get positional indexing. Measure by maximum key and you get sorted lookup. Measure by minimum priority and you get a priority queue. Measure by maximum interval endpoint and you get interval search. Measure by newline count and you get a text editor’s line index. Chapter 9 is entirely about this idea.

Finger-tree core obligations

Empty, singleton, endpoint, concatenation, split, index, insert, remove, and range operations preserve immutable snapshot semantics. Measure policies supply an identity, an element measure, and an associative combine. Split and locate depend on a documented predicate boundary over prefix measures, and any new measure or predicate helper must document the ordering assumptions it needs. Product and sum measures preserve component order and component-specific predicate mapping. Lazy middle publication and cached measures must never expose partially initialized data to a reader.

Checkpoint ports and the lazy spine

The C#, C++, and C implementations are the mature ones. Several other ports build their sorted collections, priority queues, and interval trees over a measured AVL or similar structure rather than a lazy-spine finger tree, and their API notes name the remaining parity boundary explicitly. The observable semantics

— ordering, representative retention, no-op identity, failure atomicity — are identical; the asymptotic claim for a specific operation may be weaker, and where that is so, it is written down rather than inferred.

Measures: Monoids as Connective Tissue

Give a tree an associative summary and it will answer questions you never built it to answer.

This chapter has no data structure of its own. It describes the abstraction that makes a third of the library possible, and it is the single best return on conceptual investment in this book.

9.1 The idea

A *measure* is a monoid — a type with an identity element and an associative binary combine — plus a function from an element to a value of that type. Cache the measure of every subtree on its root node, and two things become cheap:

1. **Aggregate queries.** The measure of the whole sequence is the measure at the root, read in $O(1)$. The measure of a range is a logarithmic boundary computation.
2. **Measure-guided descent.** Walk from the root, accumulating the running prefix measure. At each node, ask whether the prefix *including* the left subtree satisfies a predicate. If yes, descend left; if no, add the left measure to the accumulator and descend right. You have located the first position where the predicate flips, in $O(\log n)$, without visiting the elements you skipped.

Associativity is what makes step 2 sound — it lets the accumulated prefix be computed in any grouping. Commutativity is *not* required, and the library is scrupulous about this: ordered measures are combined strictly left to right, and several tests exist specifically to prove that a non-commutative measure behaves.

What a measure policy must supply

An identity value; an element measure; an associative combine. Nothing more. Predicates used for split and locate must be monotone over the running prefix measure — false, then true, with no oscillation — because descent is a binary search and an oscillating predicate makes the answer meaningless. Collections cannot repair an unlawful policy, and such policies are outside the logical contract.

9.2 The measures that ship

Measure	What it turns the tree into
Size / count	A positionally indexed sequence: split at “prefix count $> i$ ” gives element i .
Sum	A prefix-sum sequence; range sums by boundary subtraction where the element type allows it, or by ordered range measure where it does not.
Maximum	An ordered structure: split at “prefix max $\geq k$ ” is a sorted lower bound. Underpins sorted sets, bags, and maps.
Minimum	A priority queue: the root measure is the front priority, and descent locates it.
Order statistic (count + last key)	Sorted collections that also support rank and select.
Interval (count, last low, max high)	An interval tree: a subtree whose maximum high endpoint is below the probe cannot contain an overlap, so it is pruned.
Newline	A text rope: line count in the measure turns line/column navigation into a descent.
Popcount	The chunked bit set: inclusive rank and zero-based select by measured descent over 64-bit words.
Product / pair	Two measures carried together, preserving component order and mapping predicates to the right component. This is how “sorted <i>and</i> indexable” works.

9.3 How the nine ports spell it

The abstraction is the same everywhere; only its encoding is local. C#, C++, Kotlin, and Python expose typed measure abstractions (with comparison and predicate abstractions where their surfaces need them); C exposes equivalent policy callbacks with context pointers; Haskell uses an ordinary type class and plain functions $v \rightarrow \text{Bool}$ as predicates, because in Haskell a predicate abstraction would be ceremony. Rust exposes a `MeasurePolicy` trait plus a generous set of ready-made measures — `SizeMeasure`, `SumMeasure`, `MaxMeasure`, `MinMeasure`, `KeyMeasure`, `ProductMeasure`, `OrderStatisticMeasure`, and the combined `SizeAndSum/SizeAndMax/SizeAndMin` forms.

Where measures paid off unexpectedly

The chunked bit set was originally imagined as a HAMT-backed structure, and it half-failed the library's own test for whether a composition is load-bearing. The recommended prototype instead put `ulong` chunks at the leaves of a measured tree with a popcount sum measure — and got membership, inclusive rank, select, and reference-short-circuiting set algebra out of machinery that already existed. That is the measure framework doing what it is for: a new public capability with no new core.

RRB Vectors

The persistent vector everybody means when they say “persistent vector”.

Specimen — RrbVector

Purpose	A persistent sequence with <i>uniform</i> random access — no U-shape — plus logarithmic concatenation and splitting.
Core	Relaxed radix-balanced tree: 32-way branches, 32-element leaves. Regular branches are radix-indexed with pure bit arithmetic and carry no size table; only branches touched by concatenation carry cumulative sizes.
Index	$O(\log_{32} n)$ with a small constant, uniform across positions.
Endpoints	$O(\log_{32} n)$ — there is deliberately no persistent tail buffer.
Concat	$O(\log_{32}(n + m))$ boundary-spine merge.
Builder	Append-only, freezes full leaves safely and caches immutable snapshots.
Ports	All nine.

Bagwell and Rompf’s relaxed radix-balanced tree (2011, with the practical transient treatment in Stucki et al., ICFP 2015) is the structure behind Scala’s persistent vector, Clojure’s `core.rrb-vector`, C++ `immer`, and Rust `im`. Its central trick is to be a radix trie *most* of the time.

10.1 Regular and relaxed branches

In a plain radix trie of branching factor 32, finding index i is pure bit arithmetic: take five bits of i per level and you are there. No size tables, no comparisons. That works only if every node is full, which stops being true the moment you concatenate two vectors.

RRB's answer is to allow both kinds of node (Figure 10.1). A *regular* branch is full and radix-indexable; it stores no size table at all. A *relaxed* branch carries a small array of cumulative sizes and is searched rather than computed. Concatenation introduces relaxed branches only along the seam; everything away from the seam stays regular and stays fast.

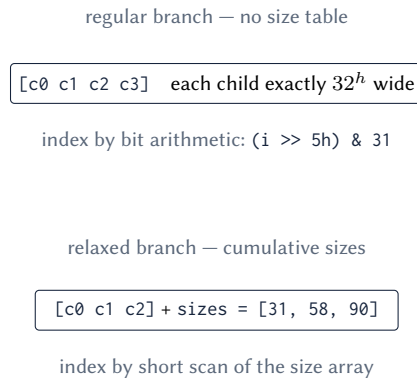


Figure 10.1: RRB nodes come in two flavors. Only nodes disturbed by concatenation pay for a size table; the rest keep the radix trie's pure bit-arithmetic indexing.

10.2 The representation boundary, stated up front

Boundary-spine concatenation, not global redistribution

All ports deliberately implement *boundary-spine* concatenation: the two child arrays at the seam are repartitioned, and nothing is promised about minimum fullness for leaves or branches away from that seam. The full Stucki-et-al. redistribution pass is not performed.

The adversarial density assertions in the test suites are regression gates for the histories they exercise — not a validator-certified representation

contract. What the validators *do* certify is a size-derived height cap: $\lfloor (\text{bit width of the count type} - 1)/5 \rfloor + 1$. The first term is the greatest minimum height required anywhere in the count domain; the extra level admits the boundary-only contract’s legal “minimum height + 1” slack even in the top count band. That works out to seven for the C#/Kotlin `Int` count domain and thirteen on 64-bit `size_t/Int/usize` targets. TypeScript and Python apply their own checked runtime count domains.

10.3 Choosing between three sequences

The library ships three persistent sequences with overlapping capabilities, and declines to pretend one of them wins.

Workload	Finger-tree deque	Rope	/	RRB vector
Endpoint push/pop	$O(1)$ amortized — best	$O(\log n)$, staged amortized	$O(1)$ / $O(\log_{32} n)$, no persistent tail	
Random index, middle	$O(\log n)$, U-shaped	$O(\log n)$ chunked	/ $O(\log_{32} n)$, small constant — best	
Concatenation	$O(\log \min)$ — best	$O(\log \min)$	/ $O(\log n)$ with relaxed nodes	
Cache density	pointer-heavy	chunked, best for scans	/ 32-way arrays, good	
Bulk build	element-wise	chunk-staged builder — best	/ builder tier	

The RRB vector’s differentiator is uniform indexing. The shipped persistent value deliberately has *no* dedicated tail buffer, so endpoint operations remain logarithmic and the separate append-only builder is the bulk-construction path. Concatenation is logarithmic but with worse constants than the deque’s.

The library’s stance is unusually restrained here: it ships the vector, ships benchmark comparisons against the rope and the platform’s own immutable list, and tells consumers to choose on their measured workload rather than on the asymptotics. Shipping an implementation does not, by itself, erase the overlap with two existing sequence families.

Ropes and Text

A string that does not mind being edited in the middle.

Specimen — Rope, MeasuredRope, TextRope

Purpose	Persistent chunked sequences (Rope), the same with a custom split/locate measure (MeasuredRope), and a newline-aware text specialization (TextRope).
Core	Runs of elements held contiguously in immutable leaf chunks at the leaves of a measured tree.
Edits	$O(\log n)$ plus bounded chunk work.
Concat / split	$O(\log \min) / O(\log n)$.
Scans	Array speed within a chunk — the reason to prefer a rope over a pointer-per-element sequence.
Ports	All nine, each with positional, measured, and text cursors.

The rope's insight is that the atoms of a sequence do not have to be its elements. Put a run of elements in one contiguous immutable leaf, and you get tree behavior for edits with array behavior for scans — which is exactly the trade a text editor wants, since editing is local and rendering is sequential.

11.1 Three layers, one machine

Rope<T> is positional. Its measure is element count; indexing, splitting, insertion, removal, concatenation, and slicing are all defined over logical element offsets.

MeasuredRope<T, M> replaces the count measure with a caller’s monoid, so split and locate can be driven by anything associative. It inherits the measure-law requirements of Chapter 9 wholesale.

TextRope is the measured rope specialized to a newline measure, plus the helpers that make text navigation pleasant: line and column conversion, line extraction, newline-style handling.

Because **TextRope** is a specialization rather than a separate core, its line index is not a side table that can drift out of sync — it is the measure, maintained by the same code path that maintains everything else.

What a text position counts

This is the least portable thing in the library, and the docs are correspondingly blunt about it. Text positions count: UTF-16 code units in C#, Kotlin, and TypeScript; char-sized bytes in C and C++; Haskell `Char` elements; Unicode scalar values in OCaml and Rust; Unicode code points in Python. *None* of them denotes a grapheme cluster index. Line and column rules are zero-based, and the C port uses LF-only newline recognition.

If you are writing cross-language code that exchanges text offsets, this is the paragraph that will save you.

11.2 Builders

Ropes are commonly built from a stream rather than edited into existence, so every port has a builder that stages chunks and publishes an immutable rope. The contract that matters is publication isolation: later builder changes cannot affect a previously produced rope. As with the trie builders, mutation is confined to nodes nobody can see.

The Range-Update Sequence

Lazy propagation, but you are allowed to keep the old version.

Specimen — RangeUpdateSequence

Purpose	An immutable indexed sequence that can apply one algebraic transformation to a contiguous range, and query that range's aggregate, without visiting the covered elements.
Core	A separately owned path-copied implicit AVL — <i>not</i> a tagged modification of either finger-tree engine.
Whole-sequence update	$O(1)$: one new root.
Subrange update / query	$O(\log n)$ boundary work.
Point edit, split, concat	$O(\log n)$.
Policy	A tag monoid acting lawfully on elements and on cached ordered measures, with an explicit composition direction.
Ports	All nine. OCaml ships an immutable-array checkpoint with the same public algebra and persistence semantics and makes no implicit-AVL lazy-update claim.

Competitive programmers know the ephemeral version of this structure as a segment tree with lazy propagation, and it is one of the great practical algorithms: “add 7 to every element between positions 200 and 9,000” in logarithmic time, by writing a pending

instruction on a handful of internal nodes instead of touching nine thousand elements.

Making it persistent is harder than it sounds, because the classic implementation *pushes* pending tags down as it descends — and pushing is mutation. In a persistent structure a read must not mutate anything, because another version is looking at those same nodes.

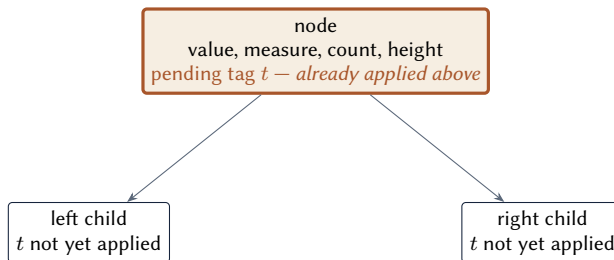
12.1 The invariant that makes it work

The library resolves this with a carefully chosen convention (Figure 12.1):

The node invariant

A node's stored value and cached measure *already include* that node's own pending tag. Its children do *not*.

Structural descent and rotations push a pending tag down before rearranging nodes. Read-only indexing, range measurement, and enumeration instead carry the inherited tag down the path and compose it after the node's older tag — never writing to shared storage.



Reads carry t downward in an accumulator and compose it with each child's own tag. Nothing is written.

Structural writes push t into both children first, then rotate or rebuild — on freshly copied nodes.

Figure 12.1: The pending-tag convention. Because a node's own summary already reflects its tag, a whole-subtree transformation is a single-node path copy, and cached measures stay queryable without ever visiting an element.

The payoff is direct. Applying a non-identity tag to the entire sequence allocates exactly one new root — $O(1)$, regardless of length. A proper subrange decomposes into

$O(\log n)$ maximal covered subtrees plus two boundary paths, so it is logarithmic. And because a node's cached measure is already tag-inclusive, range aggregate queries are logarithmic too, with no element visits at all.

12.2 The algebra, and its laws

The policy — `IRangeUpdateAlgebra<TElement, TMeasure, TTag>` in C#, with local spellings elsewhere — extends the ordinary measure monoid with a tag monoid and two actions. The single most important detail is the composition direction, and it is stated everywhere it appears:

`Compose(newer, older)` means: apply *older* first, *newer* second.

The executable law gate covers the following, and a policy that violates them is outside the collection's logical contract — the structure cannot repair an unlawful action and does not pretend to:

- ▶ the measure has an identity and an associative ordered combine;
- ▶ tags have an identity and an associative composition in that direction;
- ▶ the tag action on an element composes in the same order;
- ▶ the tag action on a cached measure composes in the same order *and receives the represented element count*;
- ▶ applying a tag to a singleton measure agrees with measuring the tagged element;
- ▶ the action distributes over ordered measure combination with the correct left and right counts — tested with a non-commutative measure;
- ▶ applying any tag to the empty measure at count zero yields the empty measure; and
- ▶ every tag that `IsIdentity` recognizes — including a value-distinct representation of identity — obeys the full identity equations.

12.3 In use

The canonical example is an affine algebra whose tags either assign a value or add to it, over a sum measure:

```
using Sequence = Durable7.FingerTree.RangeUpdateSequence<
```

```

long, long, AffineTag, AffineSumAlgebra>;

var source    = Sequence.Create([1, 2, 3, 4]);
var added     = source.ApplyRange(1, 2, AffineTag.Add(10));
// [1, 12, 13, 4], Measure == 30

var assigned = added.ApplyRange(2, 2, AffineTag.Assign(7));
// [1, 12, 7, 7], Measure == 27

long middleSum = assigned.MeasureRange(1, 2);    // 19
var (prefix, suffix) = assigned.SplitAt(2);
var reconstructed = prefix.Concat(suffix);

// source is still [1, 2, 3, 4].

```

Note what the API deliberately does *not* have: there is no `AddRange` or `AssignRange` method on the generic core, because those verbs belong to a particular tag algebra rather than to the sequence. The core knows about tags; it does not know what they mean.

Behavioral obligations

Retained versions stay immutable and share unaffected interiors. Range validation precedes tag callbacks; empty ranges preserve the receiver and the empty measure without invoking the policy at all; recognized identity updates preserve the receiver. Direct insert, replace, and remove push old tags before installing new content, so a prior range tag never leaks onto newly inserted elements. Any policy exception publishes no successor.

Enumerator copying

The concrete enumerator is a mutable struct that is deliberately *not* safe to copy mid-traversal: copies share traversal state and the stale copy fails fast once the other advances. Independently created enumerators are fully independent and safe for concurrent reads. This is the standard trade for a non-allocating enumerator, and the fail-fast behavior exists so that the mistake surfaces immediately rather than as silent misreads.

The Reversible Deque

Reversal as a fact about how you are looking, not about where things are.

Specimen — ReversibleDeque

Purpose	A persistent sequence for which logical reversal is a primary operation.
Core	Persistent deque storage plus an orientation bit or mirrored root.
Reverse	$O(1)$ — a new view over shared storage, not a materialized sequence.
Everything else	Endpoint operations, indexing, splitting, concatenation, traversal, and materialization all observe the current logical orientation.
Ports	All nine.

This is the smallest structure in the book and it makes a point worth more than its size: a great many “operations” are really changes of perspective, and a persistent library is in an unusually good position to represent them as such.

```
var deque    = ReversibleDeque<int>.Create(1, 2, 3);
var reversed = deque.Reverse();

var first    = reversed[0];           // 3
var restored = reversed.Reverse(); // shares storage with deque
```

Orientation obligations

Reversal returns a new value and preserves the original. Every operation — endpoints, indexing, splitting, concatenation, traversal, materialization — observes the logical orientation. Mixed-orientation concatenation must not silently reinterpret one operand in physical order. An $O(1)$ reversal claim belongs only where the implementation genuinely uses an orientation bit, mirrored root, or equivalent view rather than materializing a new sequence.

That last clause is the interesting one. It would be easy for a port to implement `Reverse` by building a reversed copy, keep every observable behavior identical, and quietly break the one promise anybody chose the type for. The repository maintains a dedicated complexity audit for exactly this family, and it is the required reading before touching any orientation-aware implementation.

DABA Lite: The Deliberate Outlier

Not persistent, not sorry.

Specimen — DabaLite

Purpose	Maintain the aggregate of a FIFO sliding window under any associative monoid — no commutativity, no inverse required.
Core	The De-Amortized Banker’s Aggregator (Lite), over a linked queue of 64-slot chunks with six ordered cursors.
Callback ceilings	Insert invokes Combine at most three times; evict at most twice; query at most once; an empty query, none. Unconditional.
Operations	Worst-case $O(1)$ structural work, given $O(1)$ monoid callbacks.
Mutability	Mutable and unsynchronized. This is the one collection in the book that is not persistent.
Ports	All nine.

Every other structure in this book preserves its old versions. This one overwrites them on purpose, and it belongs here anyway for two reasons: it is the measure framework’s first consumer outside the trees, and it answers a question the trees answer worse.

The question is: what is the sum — or max, or concatenation, or any associative combination — of the last k items in a stream, updated as items arrive and depart? A finger tree answers it in $O(\log n)$. Tangwongsan, Hirzel, and Schneider’s aggregator (DEBS 2017, with the Lite representation in the 2021 journal article) removes the logarithm entirely for this deliberately mutable special case.

14.1 Six cursors and a bounded fixup

The window is a queue of partial aggregates, overlaid with six cursors that always satisfy $F \leq L \leq R \leq A \leq B \leq E$ (Figure 14.1). Between them they delimit the front, left, right, accumulator, and back regions; two extra fields carry the products that do not live in queue slots.



Every insertion or eviction performs exactly *one* bounded fixup: front exhaustion collapses the cursor state, $L = B$ initializes the next flip, $L = R$ advances the three work cursors, and otherwise one left and one right partial aggregate are rewritten. There is no unbounded reversal loop — that is the “de-amortized” in the name.

Figure 14.1: The DABA Lite cursor state. The schedule is what makes the operation bounds worst-case rather than amortized.

The invocation ceilings — three, two, one — are unconditional and tested directly by counting callbacks. What is *not* unconditional is the wall clock: the complete operations are worst-case $O(1)$ only when the monoid’s `Combine` and `identity` are themselves $O(1)$. The library states the ceiling and the condition separately, which is the honest way to do it.

14.2 The parts that differ by runtime

Clear is not $O(1)$ everywhere

On the tracing runtimes — C#, Kotlin/JVM, TypeScript, Python — `Clear` swaps in one fresh empty chunk in $O(1)$ and invokes `Combine` zero times.

In C, C++, and Rust it is deliberately $O(n + c)$ for n values across c chunks, because deterministic destruction has to actually release n generic owned values. All three break the chain iteratively, neither leaking nor deferring an unbounded retired representation. Insert, eviction, and query keep their worst-case $O(1)$ structural and callback work in every port.

Python is a third case: it clears by replacing the active chain in $O(1)$, but unreachable cyclic backing may be reclaimed later by the collector, so it does not

claim the native ports' prompt deterministic destruction.

Failure behavior is similarly runtime-shaped but semantically uniform: a throwing monoid callback leaves the published window unchanged for every mutating operation. C++ achieves this by building a private fixup plan under a nothrow-move value constraint and committing without throwing; Rust plans candidate aggregates, slot rewrites, and a possible successor chunk before publication so a panic cannot expose a partial mutation; C stages callback-produced values in seven fixed aligned temporaries and transfers ownership by byte move.

14.3 What it will not do

No oldest-value access, no enumeration, no synchronization

The Lite schedule intentionally overwrites raw values with partial aggregates, so there is no API to read the oldest value or enumerate the window — the information is genuinely gone. There is no synchronization either; callers must not overlap access to one instance without external serialization. And the type is *ephemeral*: it is a fit for transient streaming state, not a persistent replacement for a deque.

14.4 Space, and a validator that knows nothing

The paper's logical accounting is $n + 2$ values: n queue partial aggregates plus the two aggregate fields. The chunked implementations instead allocate capacity for n live positions plus 1 to 127 slack slots, and an empty instance retains one 64-slot block.

`ValidateStructure` is deliberately callback-free and content-blind. In $O(c)$ time and space for c active chunks it checks links, cursor reachability and order, the count and distance equations, the DABA region-size equations, and the chunk/slack bound — and returns a statistics record with the count, five region lengths, active block count, allocated capacity, and slack. It cannot check that the aggregate is *correct*, because the original values may already have been overwritten; correctness is tested against an external FIFO model instead. That division of labor is a small model of the whole repository's approach to validation.

PART IV

Order and Priority

Sorted Collections

A measured tree with a maximum-key measure is a sorted set. Everything else is presentation.

Specimen — SortedBag, SortedSet, SortedDictionary

Purpose	Immutable comparer-ordered multisets, sets, and key/value maps, with rank and boundary queries.
Core	Order-statistic measures over the measured tree — typically cached count plus last key — in the mature ports; an idiomatic ordered storage checkpoint elsewhere.
Identity	<i>Comparison</i> defines equality here, not hashing. Two values that compare equal are the same element.
Enumeration	Bag: sorted, with duplicates. Set: sorted, one representative per equivalence class. Map: sorted by key.
Builders	Batched edits published as one immutable snapshot.
Ports	All nine.

These are the collections most people reach for first, and in this library they are facades: put a measure that summarizes “the largest key in this subtree” on a measured tree, and splitting at “prefix maximum $\geq k$ ” is a sorted lower bound. Add a count to the measure — a product measure — and the same tree also supports rank and select. The sorted collections are that observation, dressed for use.

Sorted-collection obligations

Comparison policy defines equality. If two values compare equal, the API must document whether duplicates are counted, rejected, replaced, or merged — and each of the three families answers differently, which is the point of having three. Sorted bag enumeration includes duplicates in sorted order. Sorted set enumeration contains one representative per comparison-equivalent value. Sorted map lookup, rank, and boundary queries use key comparison, never hash equality. Rank and index APIs must define zero- versus one-based indexing, miss behavior, and duplicate handling. Builders must document when they stop sharing with immutable snapshots and when snapshots are published.

Comparison equality is not hash equality

A value can be equal under a comparer and unequal under an equality comparer, or vice versa, and mixing the two families in one program is a reliable source of confusion. A case-insensitive comparer makes "Key" and "key" the same element in a `SortedSet` and different elements in a `PersistentHashSet` — unless the hash policy agrees, which it will only do if someone made it agree. The library keeps the two policy kinds firmly separate and never derives one from the other.

Canonical Sorted Sets: Zip and Zip-Zip Trees

For collections compared far more often than they are modified.

Specimen — CanonicalSortedSet and ZipTreeRankPolicy

Purpose	A sorted set whose <i>shape</i> is determined by its contents under a retained rank policy, so equal sets converge on equal topology and inequality is usually rejected in $O(1)$ by a memoized digest.
Core	Zip-zip tree: a randomized-rank binary search tree where insertion and deletion are single root-to-position <i>unzip</i> and <i>zip</i> passes — no rotations.
Ranks	A geometric primary coordinate and a fixed 64-bit secondary coordinate, both derived from HMAC-SHA-256 over the equivalence class.
Policy modes	An unexposed random key, a public reproducibility seed, or a caller-retained secret key.
Equality	Memoized root digest rejects most inequality in $O(1)$; shared nodes prune the rest; independently built equal sets still cost $O(n)$.
Ports	All nine.

Tarjan, Levy, and Timmel’s zip trees (2018/2019) reformulate the treap so that insertion and deletion are single passes: to insert, *unzip* the search path into two spines and hang them off the new node; to delete, *zip* the two child spines back together. Expected restructuring is $O(1)$ and there is no rotation code at all. Gila, Goodrich, and Tarjan’s

zip-zip trees (2023) refine the rank distribution so that expected depth approaches the information-theoretic optimum.

What Durable7 wants from them is not speed. It is *canonicity*.

16.1 Shape from contents

If a node's rank is derived from its key rather than drawn fresh at insertion time, then under one retained coherent rank policy, the tree's shape is a function of its contents (Figure 16.1). Insert the same elements in a different order and you converge on the same topology.

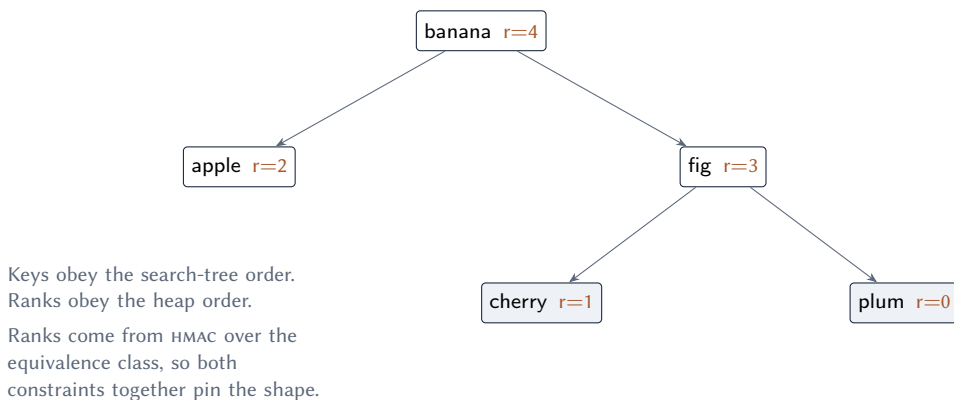


Figure 16.1: A zip-zip tree. Because ranks are derived rather than drawn, the search order and the heap order jointly determine the topology for a given content set under a given policy.

Canonical shape is worth having when a collection is compared far more often than it is modified: memoization keys, configuration fingerprints, cache validity stamps. A memoized root digest rejects most inequality in constant time; shared node identity prunes the equal case; and independently built equal sets fall back to an honest $O(n)$ comparison.

16.2 Getting ranks right

The rank source has to be stable, well distributed, and coherent with the comparer — and this turns out to be where the design gets interesting.

Rank policy obligations

A policy may use an unexposed random key, a public reproducibility seed, or a caller-retained key. An explicit comparer must *also* supply a rank hash that is constant on its equivalence classes; duplicate representatives dynamically check that contract.

Set equality is semantic across policy families, but canonical structural algebra is gated on policy identity. First-representative retention means equivalent values can still retain different concrete objects, so this is *policy-canonical* topology — not a claim of globally unique representation.

What the fixed-width variant does not inherit

This is a practical fixed-width implementation, not the paper's compact representation. Its 64-bit secondary coordinate does not inherit the paper's $O(\log \log n)$ -metadata theorem, and the 64-bit caller hash (32-bit for the fallback) can collide before HMAC ever runs. Expected-depth claims therefore require a well-distributed, stable, equivalence-class-coherent rank source.

A keyed policy resists adaptive rank prediction only while the caller-retained key stays secret; a public seed buys reproducibility, not adversarial security. And persistence changes the cost profile: an ephemeral zip tree changes $O(1)$ expected pointers, but a persistent update still copies its search/zip/unzip path — expected $O(\log n)$ nodes, $O(n)$ under adversarial ranks.

The implementations take the stack-safety problem seriously, because a fully colliding rank chain is a legitimate adversarial input: zip and unzip are explicit-stack rather than recursive, and destruction uses an intrusive non-allocating worklist so that reclaiming a 16,384-node chain neither recurses nor allocates.

Cryptographic sourcing is likewise concrete rather than hand-waved: CNG on Windows and OpenSSL elsewhere for C and C++, RustCrypto HMAC/SHA-256 with `getrandom` for Rust, `crypton` for Haskell, `hashLib` and `hmac` for Python. Rust and Python both pin stable rank hashes explicitly rather than trusting a default hasher, because `DefaultHasher` and `randomized str` hashing are exactly the failure this structure cannot tolerate.

Insertion Order as a First-Class Property

Two collections can hold the same elements and still not be the same list.

Specimen — the neutral Ordered family

Purpose	Sets, maps, and multimaps whose enumeration follows first insertion plus explicit positional operations — neither hash order nor comparison order.
Core	A persistent hash membership/stamp index composed with a persistent ordered sequence. Deliberately independent of any other family.
Identity	Equality policy decides membership and representatives. It never decides position.
Operations	Explicit movement, positional reads, ranges, reversal, stable one-shot sort, receiver-policy set algebra, all six set relations.
Ports	All nine: PersistentOrderedSet, PersistentOrderedMap, PersistentOrderedMultimap.

Insertion order is the order humans expect. It is what a configuration file means, what a UI list means, what a pipeline stage sequence means. Hash order is an implementation detail and comparison order is somebody else's opinion; if the program's meaning depends on the sequence in which things were added, that has to be represented deliberately.

17.1 Two indexes, one meaning

The family composes a hash index — which answers membership and holds representatives — with a persistent ordered sequence, which answers position. The composite obligation from Chapter 4 applies in full: both indexes publish together or neither does, counts agree, and no entry exists in only one.

Ordered-family obligations

Equality policy defines membership, duplicate collapse, stored representatives, algebra, and relations; every result retains the exact receiver policy, including empties.

Construction and duplicate additions retain the *first* representative *and its position* — a duplicate add never moves or replaces anything. Movement is explicit, retains the stored representative, and interprets a supplied destination as the final result index.

Union appends argument-only classes after receiver order; intersection and difference retain receiver order; symmetric difference emits receiver-only then argument-only classes. All algebra and all six relations eagerly enumerate and normalize the complete argument under the receiver policy *before* applying any shortcut, so a late enumeration or policy failure is visible rather than optimized away.

Documented logical no-ops return the receiver; failures publish nothing; private sparse position labels are not API, and a relabel rebuild leaves retained versions untouched.

The map adds one rule that is easy to get wrong and important to get right: setting an existing key changes *only* its payload. It does not replace the stored key representative and it does not move the entry. Range construction retains the first key representative and its first position while the last payload for that class wins. Equality policy controls key identity and never payload equality or sort order.

The multimap nests the same idea: key groups retain first-insertion order, and the distinct values within each group retain their own first-insertion order. Enumeration is grouped, and an empty group is never stored.

On sparse labels and branching

Insertion-ordered structures are usually implemented with sparse integer position labels and midpoint insertion, which occasionally require a wholesale relabel. That relabel is an $O(n)$ -at-threshold amortization — precisely the kind that Chapter 1 warns does not survive branching. The library’s response is to treat the labels as strictly private, to state the worst case per produced version rather than the amortized one, and to guarantee that a relabel rebuild leaves retained versions untouched. The related general-purpose `OrderMaintenanceList` — the Dietz–Sleator structure that answers “does A precede B ?” — remains unbuilt for want of a named consumer (Appendix C).

Priority: Three Structures, Three Reasons

“Give me the smallest one” has more variations than it looks.

Three families in this library answer priority questions, and they are not redundant — each exists because the other two are wrong for a specific need.

18.1 The measured priority queue

Specimen — PriorityQueue

Purpose	The general persistent minimum-priority queue.
Core	A minimum-priority measure over the measured tree.
Bounds	Amortized: $O(1)$ peek, $O(\log n)$ enqueue and dequeue, $O(\log \min)$ meld.
Ties	Equal priorities drain in insertion order.
Ports	All nine.

This is the default, and it is a measure facade: cache the minimum priority of each subtree, and locating the front entry is a descent. Melding two queues is concatenation. It costs nothing conceptually beyond the measure framework, which is exactly why it is the default.

```
var queue = PriorityQueue<string, int>.Empty
    .Enqueue("slow", 10).Enqueue("fast", 1).Enqueue("normal", 5);
```

```

if (queue.TryPeek(out var element, out var priority))
{
    // element == "fast"; priority == 1
}

```

18.2 The Brodal–Okasaki heap

Specimen — BrodalOkasakiHeap

Purpose	Worst-case rather than amortized latency for insert, meld, and minimum.
Core	Bootstrapped skew-binomial queues (Brodal and Okasaki, JFP 1996).
Bounds	$O(1)$ <i>worst-case</i> insert, meld, and find-minimum; $O(\log n)$ worst-case delete-minimum.
Meld	Requires comparer object identity — even when one operand is empty.
Enumeration	Unspecified structural order. Equal elements have no stable tie order. Delete repeatedly for sorted output.
Ports	All nine.

The measured queue has the same bounds *amortized*. This heap’s entire reason for existing is the word “worst-case”: if a latency budget is a hard requirement rather than an average, an amortization spike is a bug. The implementation is a direct port of the fused bootstrapped skew-binomial representation, with comparison-count tests that assert the operation bounds rather than merely believing them.

One algorithm, two spellings

The C and C++ `skew_meld` implementations consolidate the child forest by rank buckets with carry; C#, Haskell, Kotlin, and Rust spell the equivalent step as `uniquify` followed by `unionUnique`. This is an intentional native implementation-shape difference. Both produce the same valid ranked-tree multiset, the same minimum, and the same public bounds — and the repository records the di-

vergence explicitly so that a future reader does not mistake it for a porting error.

Why not strict Fibonacci or hollow heaps?

Strict Fibonacci heaps (Brodal, Lagogiannis, and Tarjan, STOC 2012) and hollow heaps (Hansen, Kaplan, Tarjan, and Zwick, ICALP 2015) achieve optimal decrease-key bounds through aggressive pointer mutation — and persistent path copying destroys exactly the surgery they depend on. They are recorded as explicit non-goals rather than as omissions.

18.3 The priority search queue

Specimen — PrioritySearchQueue

Purpose	Keyed lookup <i>and</i> minimum-priority removal <i>and</i> a combined key-range plus priority-threshold query, in one structure.
Core	A persistent AVL ordered by key, each node caching its subtree's minimum-priority winner.
Bounds	$O(1)$ minimum; $O(\log n)$ keyed update, removal, and delete-minimum; $O(\log n + v)$ range query where v counts nodes pruning cannot exclude.
Ties	Priority ties break by key order — deterministic, not merely stable.
Ports	All nine.

A heap cannot find a key. A map cannot find the minimum. The priority search queue does both, and adds the query that motivates it: *which entries in this key range have priority at most p ?* That is the scheduler question, the expiry question, and the task-queue question.

```
var jobs = PrioritySearchQueue<string, int, string>.Create(
    StringComparer.Ordinal, Comparer<int>.Default)
    .SetItem("compile", 3, "Build the solution")
    .SetItem("deploy", 8, "Publish artifacts")
    .SetItem("audit", 1, "Check invariants");
```

```

var next    = jobs.Minimum;                                // "audit", 1
var urgent = jobs.EnumerateAtMost("a", "z", maximumPriority: 3);
jobs = jobs.DeleteMinimum(out var removed);

```

A deliberate departure from Hinze

Hinze's ICFP 2001 priority search queue uses *priority-search pennants*: a loser tree over a weight-balanced search tree, whose threshold-only *at-most* operation has the output-sensitive bound $\Theta(r(\log n - \log r + 1))$ for r results.

Durable7 deliberately uses a different representation — a persistent AVL node carrying its own entry plus a cached subtree winner — and claims the bounds that representation actually delivers: $O(\log n)$ keyed operations, $O(1)$ minimum, and $O(\log n + v)$ for the inclusive key-range and priority-threshold traversal, where $v \leq n$ is the number of visited nodes that pruning could not exclude. The worst case is therefore $O(n)$ for a dense result, and no pennant-specific output bound is claimed. Choosing a simpler representation and reporting its real bounds is treated as preferable to importing a bound the code does not earn.

Intervals

Everything that has a beginning and an end wants one of these.

Specimen — IntervalTree and PersistentIntervalMap

Purpose	Ordered collections of intervals with overlap and point-stabbing queries (tree), and the same with one payload attached to each complete interval key (map).
Core	Interval annotations in the measure: typically count, last low endpoint, and maximum high endpoint.
Pruning	A subtree whose maximum high endpoint lies below the probe cannot contain an overlap, so the descent skips it entirely.
Map key	The <i>complete</i> (low, high) pair. Intervals sharing only one endpoint stay distinct.
Ports	All nine.

The interval tree is the measure framework's most elegant payoff. Put the maximum high endpoint of a subtree in its measure, and a stabbing query becomes a descent that prunes any subtree which *provably* cannot contain a match — without looking at a single interval inside it.

Interval obligations

Interval construction must define whether empty or inverted intervals are rejected, normalized, or accepted. Endpoint comparison policy defines ordering,

equality, overlap, and containment. Query APIs must document endpoint inclusivity and behavior when several overlaps exist. Removal must state whether it removes one match, all matches, or a comparison-equivalent representative.

The interval *map* adds a payload and, with it, a second index: an exact lookup index keyed by the complete interval pair, alongside the augmented search index. The composite rule applies — both contain the same intervals, payloads, and count. Replacing a payload preserves the stored interval representative and neither duplicates nor moves the key; stabbing and overlap queries return entries in the local deterministic interval order.

PART V

Authenticated Collections

The Merkle Search Tree

Structural sharing works across versions. Content addressing works across machines.

Specimen — MerkleSearchTree

Purpose	An ordered, content-addressed persistent map whose root digest names the entire collection, so two processes that agree on a digest agree on every entry.
Core	A $B = 16$ wide tree in which each key's layer is derived from the count of leading zero nibbles of its SHA-256 digest — a geometric distribution — so the shape is independent of insertion order.
Identity	Policy <code>mst-sha256-b16-v2</code> : a versioned semantic id, comparer semantics, canonical per-type codecs, and a SHA-256 domain digest.
Wire	The canonical MST2 block format — byte-identical across all nine ports.
Order	Policy comparer order, with range enumeration.
Ports	All nine, wire-compatible.

Everything up to this point exploits sharing *within a process*. Two versions of a persistent map diff cheaply because they share pointers, and pointers do not survive a network. If the two collections you want to compare live in different processes — or on different machines, or were built a week apart by unrelated code — pointer identity tells you nothing and you are back to comparing everything.

Content addressing fixes this by making the *name* of a subtree a function of its contents. Give every node a digest of what it holds, including the digests of its children, and equal digests prove equal subtrees no matter who built them or when. That is the Merkle idea, and applying it to an ordered map gives the structure behind AT Protocol repositories (Bluesky) and Dolt’s versioned SQL tables.

20.1 Shape from hashes, not from history

A Merkle tree is only useful if equal contents produce equal digests, and that requires equal contents to produce the *same shape*. An ordinary B-tree fails this immediately: insert the same entries in a different order and you get a different — equally valid — tree.

Aurolat and Taïani’s Merkle search trees (SRDS 2019) fix the shape by deriving it from the data. Hash each key; count its leading zero nibbles; that count is the key’s *layer*. Since each additional zero nibble is a 1-in-16 event, layers follow a geometric distribution with $B = 16$, which is exactly the branching behavior a B-tree wants — but assigned by the hash function rather than by insertion history (Figure 20.1).

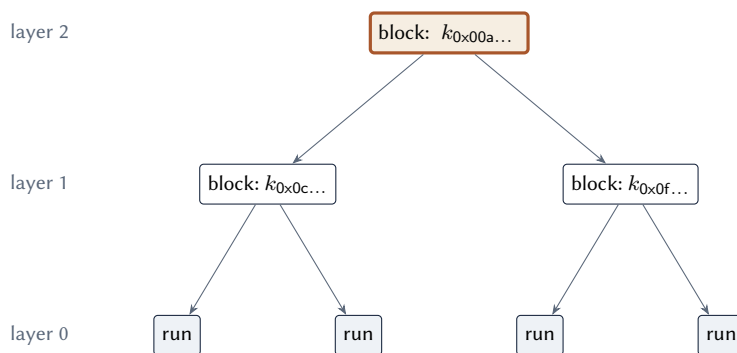


Figure 20.1: Layer assignment in a Merkle search tree. A key’s layer is the number of leading zero nibbles of its SHA-256 digest; one block holds each consecutive same-layer run, and that block’s digest covers the policy domain, its entries, its subtree count, and its child digests. Because the shape comes from the hashes, two processes that insert the same entries in different orders build the same tree and compute the same root digest.

One block holds each consecutive run of same-layer keys, and each block’s MST2 bytes cover the policy domain, its entries, its subtree count, and its child digests. The root digest therefore names the whole collection.

Prolly trees, the convergent cousin

Probabilistic B-trees — “prolly trees”, from Noms around 2016 and now Dolt — reach the same destination by a different route: instead of hashing each key to a layer, they run a rolling hash over the entry stream and place a node boundary wherever it crosses a threshold. This is *content-defined chunking*, the same technique that powers rsync-style deduplication. Both designs produce uniquely represented, content-addressed, history-independent search trees; Durable7 implements the Merkle search tree variant.

20.2 Two prerequisites, taken seriously

The repository treats content addressing as a capability that must be *earned* by satisfying two conditions, and records them as mandatory for any future content-addressed family.

Prerequisite 1 — pinned deterministic hashing

Default string hashing on .NET is randomized per process. That is a fine defense against hash-flooding and completely fatal for cross-process addressing. A Merkle policy therefore binds a *versioned semantic id*, canonical codecs, comparer semantics, and a SHA-256 domain digest. The policy id versions the application’s comparer semantics: if you change what “equal” means, you change the id, and old digests stop colliding with new ones by construction.

Prerequisite 2 — canonical serialization

A digest is only as canonical as the bytes it hashes. A strict versioned codec must *encode and decode*, and its decoder must accept exactly one canonical encoding while rejecting malformed, non-canonical, and trailing input. Codec ids end in -v followed by decimal digits. The built-ins cover integers, nullable strings, nullable byte arrays, and GUIDs.

Do not use a codec whose output depends on process-randomized hashes, culture, object identity, ambient serialization settings, or mutable state. Every one of those turns a content address into a lie.

```
var policy = MerkleSearchTreePolicy<int, string?>.Create(
```

```

    "document-index-v1",
    Comparer<int>.Default,
    MerkleCodecs.Int32,
    MerkleCodecs.UTF8String);

var baseline = MerkleSearchTree<int, string?>.Create(policy)
    .SetItem(10, "ten").SetItem(20, "twenty");
var updated = baseline.SetItem(20, "TWENTY").SetItem(30, "thirty");

MerkleDigest address = updated.RootHash;

foreach (var (key, value) in updated.EnumerateRange(15, 30))
    Console.WriteLine($"{key}: {value}");

```

Recreate the same policy id, comparer semantics, codec ids, and logical entries in any other language, and you get the same block graph and the same root address. That is not an aspiration in this repository — golden wire vectors are shared test data, and a tree built in Rust verifies against a proof produced in TypeScript.

Diff is not free just because digests exist

Equal block digests prune whole regions, and in-process Diff exploits that. But a topology-changing edit can move a block separator, and when it does, the implementation honestly falls back to ordered subtree comparison rather than pretending the digest pruning still applies. The shipped API therefore does *not* promise $O(\text{key divergence})$ for every diff. Block synchronization — Chapter 21 — is the divergence-oriented path, and it is the right tool when the two trees live in different processes.

Blocks, Proofs, Synchronization, and Merge

The tree is the easy half. Not trusting the network is the other half.

A content-addressed collection is only interesting if it can leave the process. This chapter covers the tier that lets it: storing blocks, moving them, verifying them without trust, proving individual claims about them, and merging two divergent descendants of a common ancestor.

Everything here ships in all nine languages with byte-identical wire behavior.

21.1 Blocks and stores

A *block* is one immutable node's canonical bytes, addressed by its digest. A *store* is a mapping from digest to block — the persistence boundary. The library's in-memory implementations are concurrent (an `RLock` in Python, a shared mutex in C++, a synchronized three-state store in C), and every store operation is atomic with respect to publication.

```
var store = new InMemoryMerkleBlockStore();
int blocksAdded = updated.Save(store);

var loaded = MerkleSearchTree<int, string?>.Load(
    updated.RootHash, policy, store);
```

The critical property of `Load` is what it refuses to assume.

Load never trusts the store

Loading recomputes every block digest, strictly decodes *and re-encodes* every entry to confirm canonicity, follows and validates every child reference, checks the result against the expected root, and applies a finite verification budget throughout. A store that returns tampered, malformed, non-canonical, or merely inconsistent blocks produces an error, not a tree.

The re-encode step is the one people skip. Decoding alone proves the bytes are *parseable*; re-encoding and comparing proves they are *canonical*, which is what the digest actually claims.

21.2 Packs, and the transport boundary

A *pack* is a set of blocks prepared for transport. A *complete* pack is self-contained. A *partial* pack can be completed from blocks the destination already holds — which is the common case, since two replicas of a slowly changing tree share almost everything.

```
MerkleBlockPack outbound = updated.ExportPack();

var replica = MerkleSearchTree<int, string?>.Import(
    outbound, policy, replicaStore);

Console.WriteLine(replica.RootHash == updated.RootHash); // True
```

Import verifies every supplied block *and* the requested root closure before it performs destination conflict preflight, and it commits nothing until all of that succeeds. A rejected import leaves the destination store exactly as it was.

21.3 Seven budgets

Verification of untrusted input is where authenticated data structures go wrong, and the failure mode is rarely a wrong answer — it is resource exhaustion. A malicious pack can be a decompression bomb, a cycle, a pathologically deep closure, or a codec that allocates a gigabyte from four bytes.

Bounded verification

Every port enforces *seven* positive finite budgets during verification, and the ordering of checks is itself part of the contract: query and proof-shape limits run *before* verifier allocation, hashing, or codec work. A malformed request is rejected before it can cost anything.

Local proof and synchronization construction consumes a read-only trusted topology view rather than applying network budgets or re-decoding values that are already retained. The budgets exist for the trust boundary, and they are applied at the trust boundary.

21.4 Proofs

A proof lets a party who does *not* have the tree verify a claim about it, given only the root digest. The canonical MSP2 format carries three kinds:

Membership — this key maps to this value in the tree named by this digest.

Non-membership — this key is *absent* from that tree. This is the harder and more valuable direction, and it works because the tree is ordered: the proof exhibits the adjacent entries that would bracket the key, and the structure guarantees nothing can hide between them.

Range — these are exactly the entries between these bounds, with nothing omitted.

Proof verification fixes query, shape, and envelope precedence before any decoder or codec work runs, and tampering with any part of a proof — the claimed value, a sibling digest, the envelope — produces a rejection rather than a wrong answer. The test suites verify this adversarially rather than by inspection.

21.5 Synchronization

Synchronization answers: *I have root A, you have root B, send me what I am missing.* The mechanism is a frontier walk. Compare digests from the root down; wherever they agree, an entire region is already shared and the walk stops; wherever they differ, descend. The blocks the destination lacks form the frontier, and the resulting sync pack is closure-pruned so that nothing already present is retransmitted.

The walk is iterative rather than recursive — a hostile tree should not be able to overflow a stack — and repair is likewise iterative, so a partially populated store

converges over successive rounds rather than requiring a complete restart.

21.6 Three-way merge

The final operation is the one that turns an authenticated map into a version control system: given a common ancestor and two descendants, produce a merged tree.

Merge obligations

The merge is typed and *present-null aware*: a key that was deleted in one branch and a key whose merged value is present-and-null are different outcomes, and the API distinguishes them rather than collapsing both into “no value”.

A merged tree with unresolved conflicts is never published. Conflict detection completes before any publication, so a caller either receives a complete merged tree or receives the conflicts — never a tree that is silently missing the contested keys.

That refusal to publish partial results is the same failure-atomicity rule from Chapter 2, applied where the stakes are highest. A partially merged authenticated tree would still have a valid root digest, which means it would be indistinguishable from a correct one — a wrong answer that carries a proof of its own correctness. The only safe design is to not produce it.

What this replaced

An earlier design sketch in the repository proposed a `MerkleHamt`: content addressing over the *unordered* hash trie. It remains deferred. The Merkle search tree is its ordered sibling and subsumes most of its uses while adding range queries and range synchronization — neither of which an unordered trie can offer. Should `MerkleHamt` ever be revived, it must reuse equally explicit hashing, encoding, verification-budget, and trust-boundary contracts; the prerequisites are not negotiable per family.

PART VI

Navigation

Cursors

A position that remembers which version it belongs to.

Persistent collections are excellent at whole-collection operations and awkward at local editing. Inserting one character at position 4,000 of a rope is a logarithmic descent; inserting sixteen characters one at a time is sixteen logarithmic descents, and every one of them re-walks the same path.

A cursor fixes this by being what an iterator is not: an immutable value that owns one collection version *and* a position within it, from which both navigation and editing are cheap and which can be retained and branched like any other persistent value.

What a cursor is, and is not

Every initialized cursor owns or retains one immutable logical version and a valid gap, rank, or measure boundary. Navigation returns a cursor over the *same* version. Editing returns a cursor over *one new* version and never changes the receiver, a retained ancestor, or a sibling branch. Snapshot is non-consuming. A cursor is not a detachable position, a mutable iterator, a builder, a transient, a bookmark, a rebase operation, or a live concurrent view. There is no implicit redirection to a later version and no cross-version application.

22.1 Gap semantics

A positional cursor denotes a *boundary*, not an element — one of the $n + 1$ gaps in a sequence of length n (Figure 22.1). This is the same model a text editor’s caret uses, and it is chosen for the same reason: an insertion point between two elements is a first-class thing, and a cursor that denoted an element could not represent “before everything”

and “after everything” without special cases.

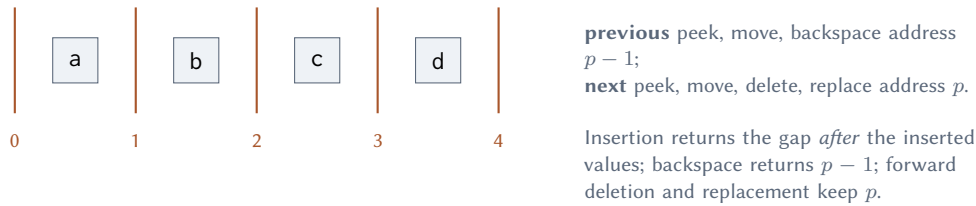


Figure 22.1: Gap semantics. Empty, start, and end gaps are ordinary first-class states rather than special cases.

Ordered cursors use the analogous idea over comparer order: the gap before the next ordered candidate. A lower-bound miss leaves a *usable* cursor rather than an error, exact search carries a separate hit discriminator, and an edit validates that the key or value actually belongs at the current gap. Nested Ordered multimap cursors keep group and value positions explicit rather than inventing a private flattened order that would not correspond to anything the collection promises.

22.2 Which families have one

Not every collection has a meaningful “next”. The library’s rule is that a cursor requires a stable *semantic* neighbor axis, and hash traversal order is not one.

Family group	Semantic axis
Patricia integer maps and sets	Ascending signed 32/64-bit key order, with rank gaps.
Measured sequences, dequeues, reversible dequeues, RRB vectors, range-update sequences, ropes and text	Position, or an ordered measure boundary. Text units remain language-local.
Sorted bags, sets, and maps; canonical sorted sets; priority search queues; interval trees and maps; chunked bit sets	Comparer order, interval order, key order, or population-rank gaps, as owned by the family.
Neutral Ordered sets, maps, multimaps	Explicit position; the multimap additionally exposes nested group and value positions.

Merkle search trees	Policy comparer order, with authenticated persistent edits.
CHAMP maps, sets, and their derived facades	<i>No public cursor.</i> The lookup and edit path is private, because hash enumeration is not semantic order.
Ctrie live views, graphs, heaps without a stable order, builders, sessions, stores, proofs, packs, DabaLite	<i>No persistent cursor.</i> No stable neighbor axis, or not a persistent aggregate at all.

22.3 One focused representation, many checkpoints

Here, once more, is the reference-versus-checkpoint pattern — and this is its most consequential instance, because the difference is a complexity claim rather than an implementation detail.

The C# positional rope cursor uses the *focused* representation selected after an explicit design comparison: a readonly struct over immutable version and navigation-context references, holding a bounded 16-element active focus with at most one partial carry of fewer than 256 elements on each side. Movement and edits return cursor values; navigation shares the logical version and its snapshot memo; an edit creates a new version state. The measured cursor extends the same model with exact ordered `MeasureBefore` and `MeasureAfter` and an absolute measure seek, preparing element measures in the cursor lineage and sharing them with descendants.

Snapshots and racing readers

A cursor created from a rope starts with that exact rope as its clean cached snapshot. A dirty first `Snapshot()` performs bounded focus and carry packing plus a canonical tree join, and publishes one winner through an interlocked compare-exchange — racing callers all receive that same rope reference. A failed candidate publishes nothing, so the cursor stays reusable. Later snapshots from any navigation cursor over the same logical version are $O(1)$ and reference-identical. Initialized cursor values are immutable and safe for concurrent reads, including concurrent first snapshots.

The proven complexity scope is a lineage, not a DAG

Creation and arbitrary seek are $O(\log n)$. Focus-local peeks, movement, and single-element edits are $O(1)$ amortized *along one linear version lineage*, with $O(\log n)$ worst-case boundary repair. Range insertion of m elements is $O(m + \log n)$ amortized. A dirty snapshot is bounded packing plus an $O(\log n)$ join; clean repeated snapshots are $O(1)$.

There is no arbitrary-version-DAG amortized bound. Editing b independently retained cursors at a carry, chunk, or lazy-spine boundary carries the conservative $O(b \log n)$ aggregate bound plus bounded per-branch copying, because potential consumed by one child cannot pay for a sibling. This is Chapter 1's branching caveat, arriving exactly where it was predicted to.

The other eight ports ship *snapshot-plus-gap* checkpoints: an already-canonical retained collection plus a validated position, delegating edits to ordinary persistent operations. Creation, navigation, positional seek, and snapshot are $O(1)$; measures, peeks, point edits, and search are $O(\log n)$ plus bounded chunk work; range insertion is $O(m + \log n)$. They preserve every observable behavior — gap semantics, branching, unconditional replacement, retained snapshots, presence-safe peeks, ordered before/after measures, absolute prefix search — and they claim no focused representation, no snapshot memo, no allocation or callback ceiling, no amortized locality, and no benchmark parity. Their API notes say so in those words.

Measured cursor result semantics, everywhere

MeasureBefore aggregates $[0, p)$ and MeasureAfter aggregates $[p, n)$; combining them *in that order* yields the whole version's measure — without assuming an inverse, commutativity, element equality, or a default-value identity.

Absolute measure seek selects the gap before the first element whose inclusive prefix satisfies a lawful monotone predicate. A predicate already true at the identity selects zero for a non-empty collection; misses and empty collections return false together with an end cursor whose before-measure is the whole measure.

A same-position seek and an empty range insertion preserve the exact stored version state. ReplaceNext deliberately does *not* consult element equality: it always creates a new logical version, and measured replacement invokes the

element-measure callback for the supplied replacement even when the object is identical.

Why cursors instead of self-adjusting structures

The classical answer to “make repeated local access cheap” is a self-adjusting structure — a splay tree, a move-to-front list. Both are rejected here for two independent reasons worth recording so the idea is not re-litigated.

First, **reads would allocate**. Adapting on read means path-copying on read: every lookup produces a new version or mutates shared state, destroying both the allocation-free read contract and thread-safe sharing of versions.

Second, **shared versions have contradictory access patterns**. Two consumers holding the same version would each want it shaped for *their* access sequence, and the amortized potential argument assumes one linear history — the same failure mode as the branching caveat above.

The cursor is the substitute: per-consumer locality, held by the consumer, without ever reshaping shared state.

PART VII

Reference



Complexity at a Glance

Bounds are for the mature reference ports unless noted. Where a port ships a checkpoint with a weaker asymptotic claim, its own API notes are authoritative — see the checkpoint caveats in the relevant chapter. Throughout, n is the receiver's size, m the argument's, c a collision-bucket or chunk count, and d a distance from the nearer end.

Structure	Operation	Bound
<i>Hash tries</i>		
PersistentHashMap / Set	lookup, insert, remove GetOrAdd / AddOrUpdate equality / diff, shared ancestry	$O(\log_{32} n)$, + $O(c)$ on collision one hash, one descent visited non-shared regions + differences
PersistentHashBag	equality / diff, independent point ops, algebra	$O(n + m)$; $O(c^2)$ per bucket as the underlying map
PersistentBiMap	add, set, remove Inverse	two map operations $O(1)$
PersistentHashMultimap	pair add / remove	$O(\log_{32} n)$
PersistentRelation	Inverse	$O(1)$ index swap
PersistentChunkedBitSet	membership, rank, select algebra	$O(\log c)$ sparse word-stream merge
<i>Integer tries</i>		
PersistentIntMap / Set	point operations union / intersect / except no-op merge	$O(\min(n, W))$ visited non-shared prefixes; $O(n + m)$ worst case reference identity preserved

Sequences

FingerTreeDeque	push / pop at either end	$O(1)$ amortized
	index, split at position	$O(\log d)$
	concat	$O(\log \min(n, m))$
RrbVector	index	$O(\log_{32} n)$, uniform
	endpoint operations	$O(\log_{32} n)$ (no persistent tail)
	concat	$O(\log_{32}(n + m))$
Rope / MeasuredRope	index, point edit	$O(\log n)$ + bounded chunk work
	concat / split	$O(\log \min) / O(\log n)$
RangeUpdateSequence	whole-sequence tag	$O(1)$, one new root
	subrange tag, range measure	$O(\log n)$
	point edit, split, concat	$O(\log n)$
ReversibleDeque	Reverse	$O(1)$
DabaLite	insert / evict / query	$O(1)$ worst case; $\leq 3/2/1$ combines
	Clear	$O(1)$ managed; $O(n + c)$ native

Ordered and priority

SortedBag / Set / Dictionary	lookup, insert, remove	$O(\log n)$
	rank, select	$O(\log n)$
CanonicalSortedSet	insert, remove	expected $O(\log n)$ path copy; $O(n)$ adversarial
	inequality via digest	$O(1)$ in the usual case
	equality, independent sets	$O(n)$
PersistentOrderedSet / Map	membership, append	$O(\log_{32} n)$ + sequence cost
PriorityQueue	positional read, move	$O(\log n)$
	peek	$O(1)$ amortized
	enqueue, dequeue	$O(\log n)$ amortized
	meld	$O(\log \min)$ amortized
BrodalOkasakiHeap	minimum, insert, meld	$O(1)$ worst case
	delete-minimum	$O(\log n)$ worst case
PrioritySearchQueue	minimum	$O(1)$
	keyed update, remove, delete-min	$O(\log n)$

IntervalTree / PersistentIntervalMap	key-range + priority query	$O(\log n + v), v \leq n$
	insert, remove, exact lookup	$O(\log n)$
	stabbing / overlap	$O(\log n + \text{reported})$, with subtree pruning
<i>Authenticated</i>		
MerkleSearchTree	lookup, insert, remove	$O(\log_{16} n)$
	root digest	$O(1)$ cached
	Save / Load	$O(\text{blocks})$, fully re-verified
	synchronize	frontier blocks only
<i>Lifecycle and navigation</i>		
Bulk builder	per entry	unpublished in-place node mutation
Editing session	adopt, publish	$O(1)$ trie work
	per edit (C# owner-token)	owned-node mutation, no path copy
Cursor (focused, C# rope)	per edit (sibling ports)	ordinary persistent path copy
	local move, single edit	$O(1)$ amortized, one lineage
	arbitrary seek	$O(\log n)$
	b retained branches at a boundary	$O(b \log n)$ aggregate
Cursor (checkpoint ports)	create, navigate, seek, snapshot	$O(1)$
	peek, point edit, measure, search	$O(\log n) + \text{chunk work}$
	range insert of m	$O(m + \log n)$



The Same Structure in Nine Languages

Names are idiomatic per port; semantics are not. Where a port's spelling differs structurally — a nested type, a module, a handle — the difference is notational.

Family	Spellings
Persistent hash map	<code>PersistentHashMap<TKey, TValue></code> (C#, Kotlin, TypeScript, Python); <code>PersistentHashMap<K, V, S></code> (Rust); <code>persistent_hash_map<Key, T, Hash, KeyEqual, ValueEqual></code> (C++); <code>HashMap k v</code> (Haskell); <code>d7_hamt_map</code> (C); <code>Persistent_hamt</code> (OCaml).
Persistent hash set	<code>PersistentHashSet<T></code> ; <code>persistent_hash_set<T, Hash, KeyEqual></code> ; <code>HashSet a</code> ; <code>d7_hamt_set</code> ; <code>Persistent_hash_set</code> .
Editing session	<code>nested Transient</code> (C#, Kotlin, C++, OCaml); <code>TransientHashMap/Set</code> (Rust, TypeScript, Python); <code>MapTransient/SetTransient</code> (Haskell); <code>d7_hamt_map_transient</code> (C).
Hash bag	<code>PersistentHashBag</code> ; <code>persistent_hash_bag</code> ; <code>HashBag a</code> ; <code>d7_hamt_bag</code> ; <code>Persistent_hash_bag</code> .
Bidirectional map	<code>PersistentBiMap</code> ; <code>persistent_bi_map</code> ; <code>BiMap k v</code> ; <code>d7_hamt_bi_map</code> ; <code>Persistent_bi_map</code> .
Integer Patricia	<code>PersistentIntMap/IntSet/LongMap/LongSet</code> ; <code>persistent_int_map</code> etc.; <code>IntMap32/IntMap64/ IntSet32/IntSet64</code> (Haskell); <code>d7_int_map</code> etc.; <code>Persistent_patricia</code> .
Concurrent trie	<code>ConcurrentHashTrie</code> (C#, Kotlin — full protocol; TypeScript, Python — facade); <code>Concurrent_hash_trie</code> (OCaml facade).
Merkle search tree	<code>MerkleSearchTree<TKey, TValue></code> ; <code>merkle_search_tree</code> ; <code>MerkleSearchTree k v</code> ; <code>d7_merkle_search_tree</code> ; <code>Merkle_*</code> modules (OCaml).

Persistent deque	<code>FingerTreeDeque<T></code> (C#); <code>PersistentDeque<T></code> (Kotlin, Rust, TypeScript, Python); <code>persistent_deque<T></code> (C++); <code>Deque a</code> (Haskell); <code>ft_persistent_deque</code> (C); <code>Persistent_deque</code> (OCaml).
Measured tree	<code>FingerTree<TElement, TMeasure, TMeasureOps></code> (C#); <code>FingerTree<T, M></code> / <code>FingerTree<T, P></code> (Kotlin, TypeScript, Rust); <code>finger_tree<Element, MeasurePolicy></code> (C++); <code>FingerTree v a</code> (Haskell); <code>ft_tree</code> (C); <code>Measured_tree</code> / <code>Measured_sequence</code> (OCaml).
RRB vector	<code>RrbVector<T></code> and <code>builder</code> ; <code>rrb_vector<T></code> / <code>rrb_vector_builder<T></code> ; <code>RrbVector a</code> ; <code>ft_rrb_vector</code> / <code>ft_rrb_builder</code> ; <code>Rrb_vector</code> .
Reversible deque	<code>ReversibleDeque<T></code> ; <code>reversible_deque<T></code> ; <code>ReversibleDeque a</code> ; <code>ft_reversible_deque</code> ; <code>Reversible_deque</code> .
Range-update sequence	<code>RangeUpdateSequence<TElement, TMeasure, TTag, TOps></code> (C#); <code>RangeUpdateSequence<T, M, Tag></code> (Kotlin, TypeScript); <code>range_update_sequence<Element, Algebra></code> (C++); <code>RangeUpdateSequence</code> (Haskell, Rust, Python); <code>ft_range_update_sequence</code> (C); <code>Range_update_sequence</code> (OCaml).
Ropes and text	<code>Rope<T></code> , <code>MeasuredRope<. . .></code> , <code>TextRope</code> / <code>RopeText</code> ; <code>rope<T></code> , <code>measured_rope<. . .></code> , <code>text_rope</code> ; <code>Rope a</code> , <code>MeasuredRope v a</code> , <code>TextRope</code> ; <code>ft_rope</code> , <code>ft_measured_rope</code> , <code>ft_text_rope</code> ; <code>Rope</code> , <code>Measured_rope</code> , <code>Text_rope</code> .
Sorted collections	<code>SortedBag<T></code> , <code>SortedSet<T></code> , <code>SortedDictionary<TKey, TValue></code> (C#); <code>SortedMap<K, V></code> elsewhere; <code>sorted_bag/sorted_set/sorted_map</code> (C++); <code>ft_sorted_multiset/_set/_map</code> (C); <code>Sorted_bag/Sorted_set/Sorted_map</code> (OCaml).
Canonical sorted set	<code>CanonicalSortedSet<T></code> and <code>ZipTreeRankPolicy<T></code> ; <code>canonical_sorted_set<T></code> and <code>zip_tree_rank_policy<T></code> ; <code>ft_canonical_sorted_set</code> and <code>ft_canonical_policy</code> (C); <code>Canonical_sorted_set</code> (OCaml).
Priority queue	<code>PriorityQueue<TElement, TPriority></code> ; <code>priority_queue<Element, Priority, Comparison></code> ; <code>PriorityQueue p a</code> (Haskell); <code>ft_priority_queue</code> ; <code>Priority_queue</code> .
Brodal–Okasaki heap	<code>BrodalOkasakiHeap<T></code> ; <code>brodal_okasaki_heap<T, Less></code> ; <code>BrodalOkasakiHeap a</code> ; <code>ft_brodal_heap</code> ; <code>Brodal_okasaki_heap</code> .

Priority search queue	PrioritySearchQueue<TKey, TPriority, TValue>; priority_search_queue<K, P, V, KeyLess, PriorityLess>; PrioritySearchQueue k p v; ft_priority_search_queue; Priority_search_queue.
Interval tree and map	IntervalTree<T>, Interval<T>, PersistentIntervalMap; interval_tree<T, Comparison>; IntervalTree a; ft_interval_tree, ft_persistent_interval_map; Interval_tree, Persistent_interval_map.
Sliding-window aggregator	DabaLite<T, TMonoid> (C#); DabaLite<T> / DabaLite<T, M>; daba_lite<T, P>; ft_daba_lite; Daba_lite.
Derived compositions	PersistentHashMultimap, PersistentRelation, PersistentOrderedMap, PersistentOrderedMultimap, PersistentMapPatch, PersistentDirectedGraph, PersistentIndexedMap, PersistentChunkedBitSet — with snake_case in C++/OCaml/Python, d7_/ft_ prefixes in C, and bare names in Haskell.

Roads Not Taken

A library is defined as much by what it declines to build. These decisions are recorded so that they are not silently re-litigated, and each carries the reason that would have to change for the answer to change.

C.1 Rejected outright

Self-adjusting structures. Splay trees, move-to-front lists, and their relatives fight persistence twice over: adapting on read means path-copying on read, destroying the allocation-free read contract; and two consumers sharing a version want it shaped for two different access sequences, so the amortized potential argument has no single history to be proved over. The substitutes are cursors and the planned frozen read-phase optimization.

Strict Fibonacci and hollow heaps. Both achieve optimal decrease-key bounds through aggressive pointer mutation, which is precisely the surgery persistent path copying destroys.

Kaplan–Tarjan real-time deques. Their $O(1)$ *worst-case* endpoints and catenation are genuinely attractive, but the shipped deque’s amortization spikes are already rare and logarithmic — memoized suspensions spread the debt — and the implementation is notoriously intricate (redundant counting systems, multiple bootstrapping layers) with constants that typically lose on real workloads. The useful deliverable instead is to *measure and publish* the shipped deque’s worst observed pause, so a future latency-critical consumer can decide with data.

C.2 Postponed, with a named gate

Size-tiered small representations. Below a threshold, represent the collection as a flat array; promote at the threshold; demote with hysteresis. Clojure’s `PersistentArrayMap` and Rust’s `im::Vector` both do it, and it targets exactly the regime where persistent maps lose worst — the three-entry map that most programs hold thousands of. It is classified as *strong but postponed*: thresholds must be benchmark-derived rather than guessed, hysteresis must prevent flapping, and representation dispatch must not tax the large case. Two supporting arguments are already recorded: a general-purpose library owns its own equality, enumeration, and no-op identity contracts and can therefore guarantee representation-blindness; and a *constant* promotion threshold costs $O(1)$ worst case, so unlike an $O(n)$ -at-threshold scheme it survives branching persistence.

The frozen collection tier. Designed, prototyped, and gated on lookup, enumeration, memory, and realistic construction break-even evidence (Chapter 7).

Key-type-specialized map construction. A factory that routes integer keys to the Patricia trie, byte-comparable keys to an adaptive radix tree, and everything else to the hash trie. The honest counterpoint is that one facade unifying three cores costs either virtual dispatch or a fat discriminated union on *every* operation, and the cheaper design is separate public types plus a documentation-level decision table. Postponed in the separate-types form; the ART core itself is a real project, gated on a consumer with genuine prefix-query or byte-ordered-key needs.

The order-maintenance list. The Dietz–Sleator structure that answers “does *A* precede *B*?” in $O(1)$ with amortized $O(1)$ insertion, via two-level integer labels and local relabeling. Plausible, and waiting for a named general consumer — dependency graphs, document anchors, CRDT position identifiers. The honest caveat is already written: relabeling amortization is a linear-history argument, so a version branched before a relabel can re-pay it, and the public type would have to state its worst case per produced version.

MerkleHamt. Content addressing over the unordered hash trie. The ordered Merkle search tree subsumes most of its uses and adds range queries and range synchronization; if revived, it must reuse equally explicit hashing, encoding, verification-budget, and trust-boundary contracts.

C.3 Shipped as documentation rather than as a type

The styled-text rope. A text rope carrying formatting runs — a measured rope for content plus a run sequence (ideally the range-update sequence, so “bold lines 10–500” is one logarithmic tag) keeping style spans aligned under edits. This is a composition showcase, and by the repository’s own parity-economics rule it ships as a *sample plus documentation pattern* rather than as a tenth family in nine languages. The editor samples are its natural home.

Further Reading

The papers behind the structures, roughly in the order the book meets them.

Topic	Source
Persistence, formally	Driscoll, Sarnak, Sleator & Tarjan, “Making Data Structures Persistent”, <i>JCSS</i> , 1989.
The functional canon	Okasaki, <i>Purely Functional Data Structures</i> , Cambridge University Press, 1998.
Hash array mapped tries CHAMP	Bagwell, “Ideal Hash Trees”, EPFL technical report, 2001. Steindorfer & Vinju, “Optimizing Hash-Array Mapped Tries for Fast and Lean Immutable JVM Collections”, <i>OOPSLA</i> , 2015.
Mergeable integer maps	Okasaki & Gill, “Fast Mergeable Integer Maps”, <i>ML Workshop</i> , 1998.
Concurrent hash tries	Prokopec, Bronson, Bagwell & Odersky, “Concurrent Tries with Efficient Non-Blocking Snapshots”, <i>PPoPP</i> , 2012.
Finger trees	Hinze & Paterson, “Finger Trees: A Simple General-Purpose Data Structure”, <i>JFP</i> , 2006.
RRB trees	Bagwell & Rompf, “RRB-Trees: Efficient Immutable Vectors”, EPFL technical report, 2011; Stucki, Rompf, Ureche & Bagwell, “RRB Vector: A Practical General Purpose Immutable Sequence”, <i>ICFP</i> , 2015.
Zip trees	Tarjan, Levy & Timmel, “Zip Trees”, 2018/2019.
Zip-zip trees	Gila, Goodrich & Tarjan, “Zip-Zip Trees: Making Zip Trees More Balanced, Biased, Compact, or Persistent”, 2023.
Worst-case meldable heaps	Brodal & Okasaki, “Optimal Purely Functional Priority Queues”, <i>JFP</i> , 1996.

Strict Fibonacci heaps	Brodal, Lagogiannis & Tarjan, <i>STOC</i> , 2012. (<i>Recorded rejection.</i>)
Hollow heaps	Hansen, Kaplan, Tarjan & Zwick, <i>ICALP</i> , 2015. (<i>Recorded rejection.</i>)
Priority search queues	Hinze, “A Simple Implementation Technique for Priority Search Queues”, <i>ICFP</i> , 2001.
Sliding-window aggregation	Tangwongsan, Hirzel & Schneider, <i>DEBS</i> , 2017; and the <i>VLDB Journal</i> , 2021, article introducing the Lite representation.
Merkle search trees	Auvolat & Taïani, “Merkle Search Trees: Efficient State-Based CRDTs in Open Networks”, <i>SRDS</i> , 2019.
Real-time dequeues	Kaplan & Tarjan, “Purely Functional, Real-Time Deques with Catenation”, <i>JACM</i> , 1999. (<i>Recorded rejection.</i>)
Order maintenance	Dietz & Sleator, 1987; Bender et al., “Two Simplified Algorithms for Maintaining Order in a List”, 2002. (<i>Postponed.</i>)
Adaptive radix trees	Leis, Kemper & Neumann, “The Adaptive Radix Tree”, <i>ICDE</i> , 2013. (<i>Postponed.</i>)

Within the repository itself, the authoritative documents are the *data-structure catalog*, the *semantic contracts reference*, the *frontier* and *derived structure catalogs*, and each workspace’s own API specification, header, and validation guide.

Index

- amortization
 - under branching, 5
- bijection, 27
- block, 95
- Brodal–Okasaki heap, 84
- BrodalOkasakiHeap, 84
- bulk builder, 40
- CanonicalSortedSet, 75
- CHAMP
 - bitmap split, 18
 - canonical deletion, 18
- checkpoint port, 7
- collision bucket, 20
- composition-first families, 25
- ConcurrentHashTrie, 35
- content addressing, 92
- Ctrie
 - generation stamping, 36
- cursor, 101
 - focused representation, 103
 - gap semantics, 101
 - snapshot-plus-gap, 104
- DabaLite, 67
- digit, 47
- directed graph, 29
- editing session, 40
- failure atomicity, 11
- finger tree, 47
- FingerTree, 47
- FingerTreeDeque, 47
- frozen collection tier, 42
- GCAS, 36
- HAMT, 18
- hash trie, *see* CHAMP
- insertion order, 79
- interval tree, 87
- IntervalTree, 87
- lazy propagation, 62
- map patch, 28
- measure, 51
- MeasuredRope, 59
- Merkle search tree
 - layers, 92
- MerkleSearchTree, 91
- monoid
 - associativity, 51
- multimap, 28
- multiset, 26
- no-op identity, 11
- Ordered family (insertion-ordered), 79
- ordering contracts, 12
- orientation, 65
- owner-token transient, 41
- pack, 96
- path copying, 3
- Patricia trie
 - maps and sets, 31
 - path compression, 32
- pending tag, 62
- PersistentBiMap, 26

- PersistentChunkedBitSet, 30
- PersistentDirectedGraph, 29
- PersistentHashBag, 25
- PersistentHashMap, 17
- PersistentHashMultimap, 27
- PersistentHashSet, 17
- PersistentIndexedMap, 29
- PersistentIntervalMap, 87
- PersistentIntMap, 31
- PersistentIntSet, 31
- PersistentMapPatch, 28
- PersistentOrderedMap, 79
- PersistentOrderedMultimap, 79
- PersistentOrderedSet, 79
- PersistentRelation, 27
- policy, 10
- population count, 30
- priority queue, 83
- priority search queue, 85
- PriorityQueue, 83
- PrioritySearchQueue, 85
- proof, 97

- radix trie, 56
- RangeUpdateSequence, 61
- rank policy, 76
- RDCSS, 36
- reference pruning, 19
- representative
 - first-representative rule, 11
- ReversibleDeque, 65
- Rope, 59
- rope, 59
- RRB vector
 - regular branch, 56
 - relaxed branch, 56
- RrbVector, 55

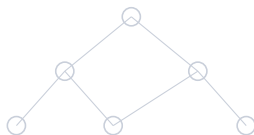
- secondary index, 29
- single-descent factories, 21
- sliding window, 68

- sorted collections, 73
- SortedBag, 73
- SortedDictionary, 73
- SortedSet, 73
- structural merge, 33
- structural sharing, 4
- synchronization, 97

- tag algebra, 63
- text rope, 60
- TextRope, 59
- three-way merge, 98
- tomb node, 36

- version DAG, 4

- zip tree, 75
- zip-zip tree, 76
- ZipTreeRankPolicy, 75



THE DURABLE7 FIELD GUIDE

Every operation returns a new version
and leaves the old one untouched.